



第八章 密文集合运算

汇报目录

「01」

知识点一：基础知识

「02」

知识点二：朴素哈希PSI

「03」

知识点三：Google PSI

「04」

知识点四：非平衡PSI优化

「05」

知识点五：基于OPRF的PSI

1.隐私集合交集

- ❑ PSI: Privacy Set Intersect, PSI是安全多方计算的一个子类
- ❑ 概念: 两方隐私保护集合交集协议允许持有各自集合的两方来共同进行两个集合的交集计算。协议最后的结果是一方或是两方得到正确的交集内容, 并且保证不会得到交集以外另一方集合中的任何信息。

- ❑ 场景:

条件: Alice拥有一个集合 $A = \{a_1, a_2, \dots, a_m\}$,

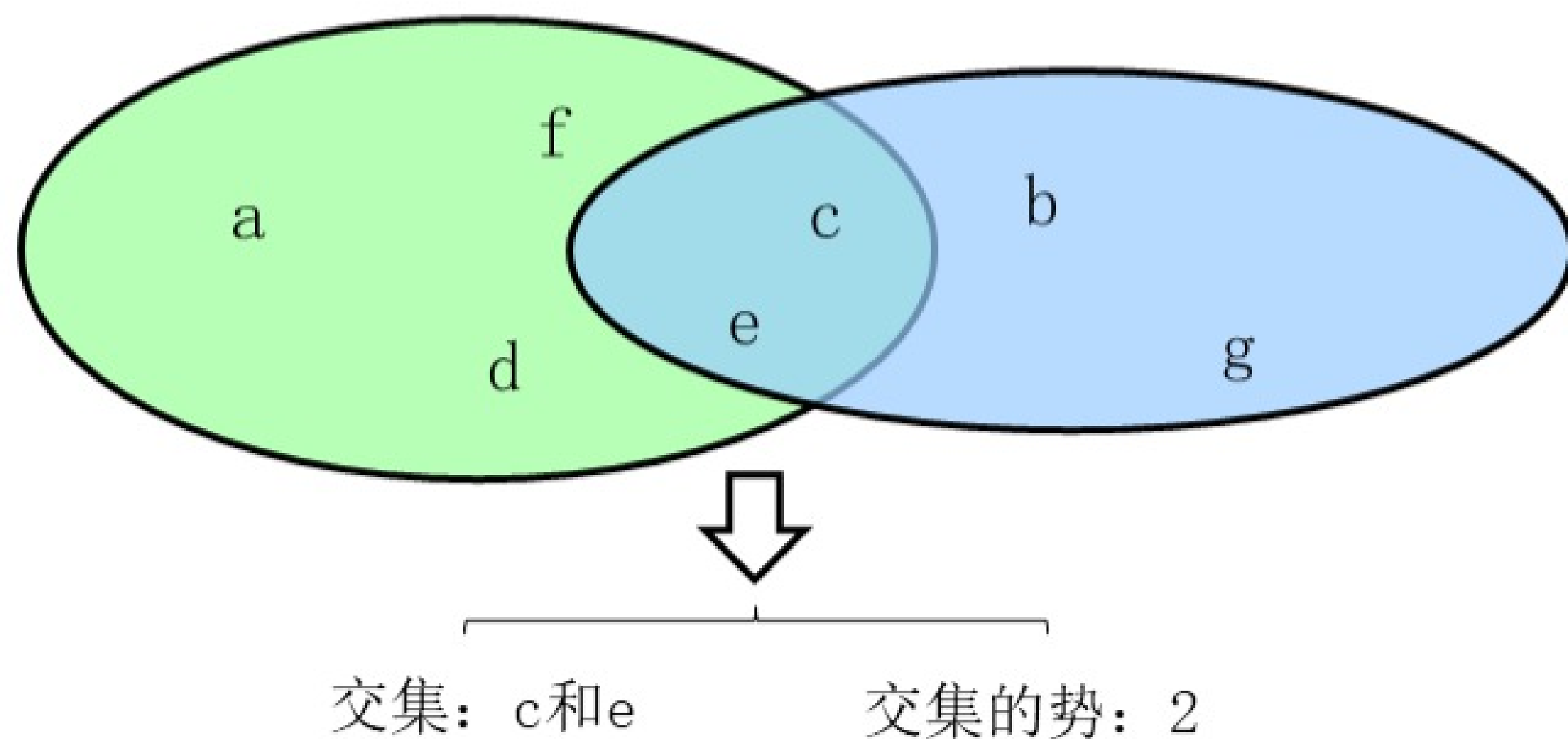
Bob拥有一个集合 $B = \{b_1, b_2, \dots, b_n\}$,

问题: 求得交集 $I = \{x; x \in A \cap B\}$

要求: 对于不属于交集I的元素, Alice不能获得集合 $B - (A \cap B)$ 的元素,

Bob也不能获得集合 $A - (A \cap B)$ 的元素。

1.隐私集合交集



- 适用于广告效果转化的计算。一方是网购平台，一方是广告平台。通过计算交集或交集的大小来完成计算。

2. 集合运算相关

□ 两类场景：

平衡PSI：两方集合数量大体相等。

非平衡PSI：两方集合元素个数差异很大，具体表现在一大一小。

□ 其他密文集合运算

并集：多方集合所有不相等元素的集合。

基数：多方集合所有不相等元素的个数。

2. 集合运算相关

□ 两类场景：

平衡PSI：两方集合数量大体相等。

非平衡PSI：两方集合元素个数差异很大，具体表现在一大一小。

□ 其他密文集合运算

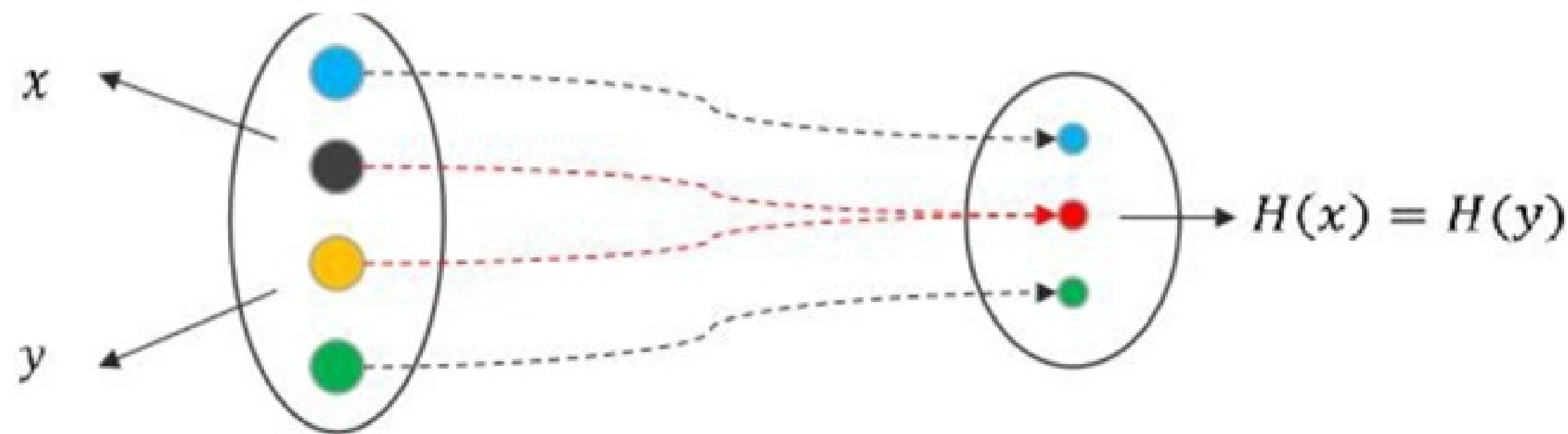
并集：多方集合所有不相等元素的集合。

基数：多方集合所有不相等元素的个数。

3. 基础知识

1. 哈希函数

- 把任意长度的输入通过散列算法变换成固定长度的输出，该输出就是散列值。这种转换是一种压缩映射，也就是，散列值的空间通常远小于输入的空间，不同的输入可能会散列成相同的输出，所以不可能从散列值来确定唯一的输入值。
- 对不同的关键字可能得到同一散列地址，即 $key_1 \neq key_2$ ，而 $f(key_1) = f(key_2)$ ，这种现象称**碰撞**。
- 单向性：通过输入可以得到输出；但通过输出无法反推输入。



任意长度的输入

固定长度 m 位的输出

3. 基础知识

- 2. 布隆过滤器

它实际上是一个很长的二进制向量和一系列随机映射函数，可用于检索一个元素是否在一个集合中。它的优点是空间效率和查询时间都比一般的算法要好的多，缺点是有一定的误识别率和删除困难。

- 原理：

初始化：

随机映射函数：Hash₁、Hash₂、Hash₃

二进制向量

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

插入Tencent

Hash₁(Tencent) mod 8=1
Hash₂(Tencent) mod 8=5
Hash₃(Tencent) mod 8=7

0	1	0	0	0	1	0	1
0	1	2	3	4	5	6	7

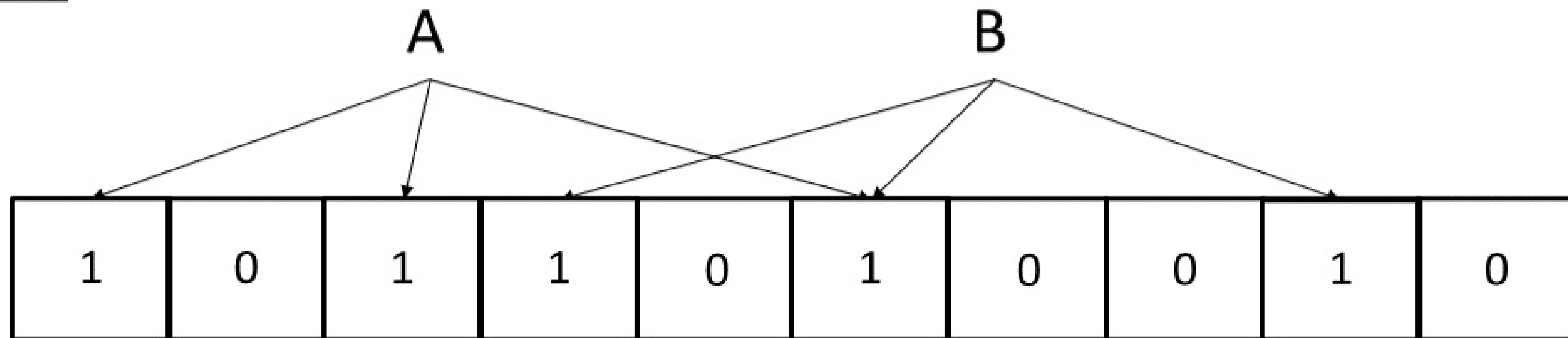
插入NKU

Hash₁(NKU) mod 8=2
Hash₂(NKU) mod 8=4
Hash₃(NKU) mod 8=7

0	1	1	0	1	1	0	1
0	1	2	3	4	5	6	7

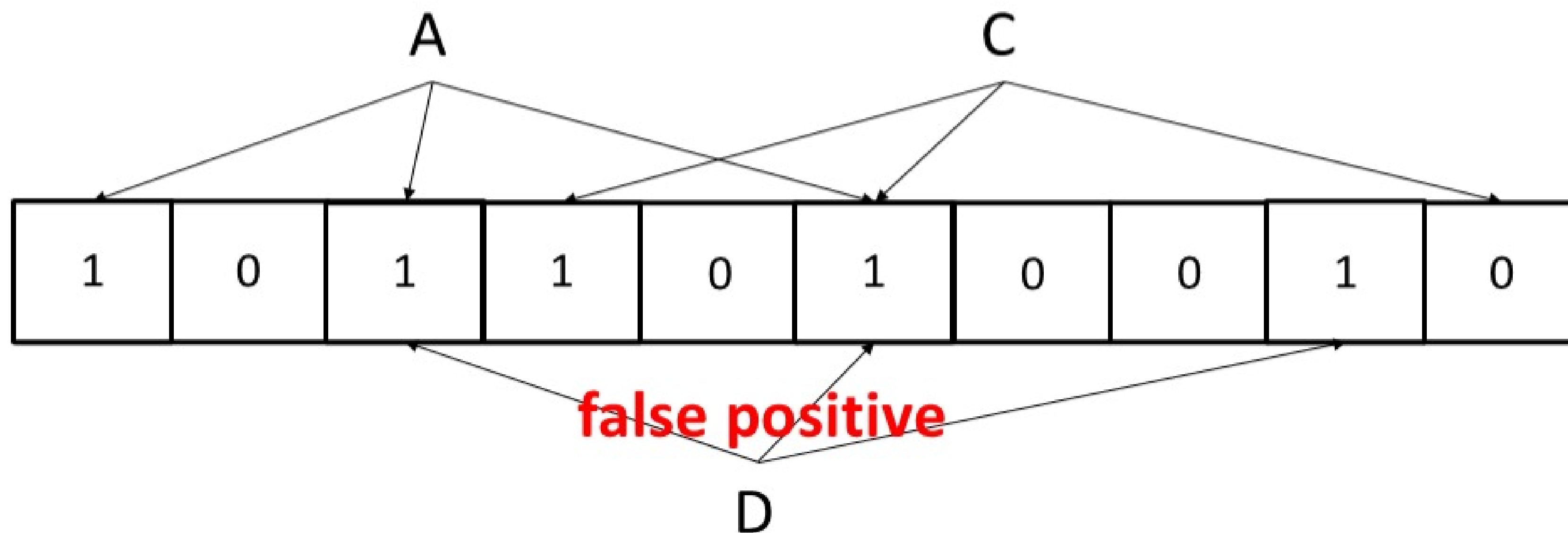
3. 基础知识

- 布隆过滤器：它实际上是一个很长的二进制向量和一系列随机映射函数，可用于检索一个元素是否在一个集合中。它的优点是空间效率和查询时间都比一般的算法要好的多，缺点是有一定的误识别率和删除困难。
- 示意：



3. 基础知识

- **用途：**可以验证一个元素是否在目标集合中。
- **假阳性：**



当元素个数 n 与比特位数 m 一定时， k 取 $\ln 2 \frac{m}{n}$ 时，假阳性率最低，误差率如表。

3. 基础知识

- 假阳率示意

布隆过滤器比特位数m与集合大小 X 的比值	假阳率
16	4.59×10^{-4}
20	6.71×10^{-5}
24	9.87×10^{-6}
32	2.10×10^{-7}
48	9.65×10^{-11}
64	4.43×10^{-14}

3. 基础知识

- 3.可交换加密： M 表示消息空间， K 表示密钥空间， 可交换加密函数 f 定义为

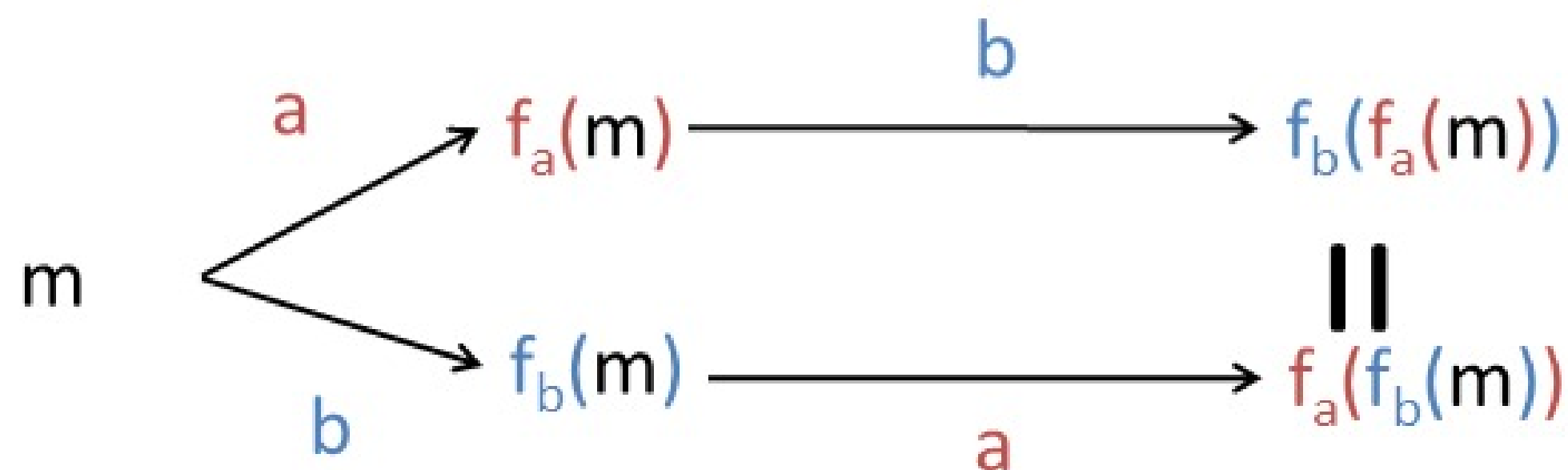
$$f: M \times K \rightarrow M$$

其中， f 表示双射关系。

可交换加密可以表示为：

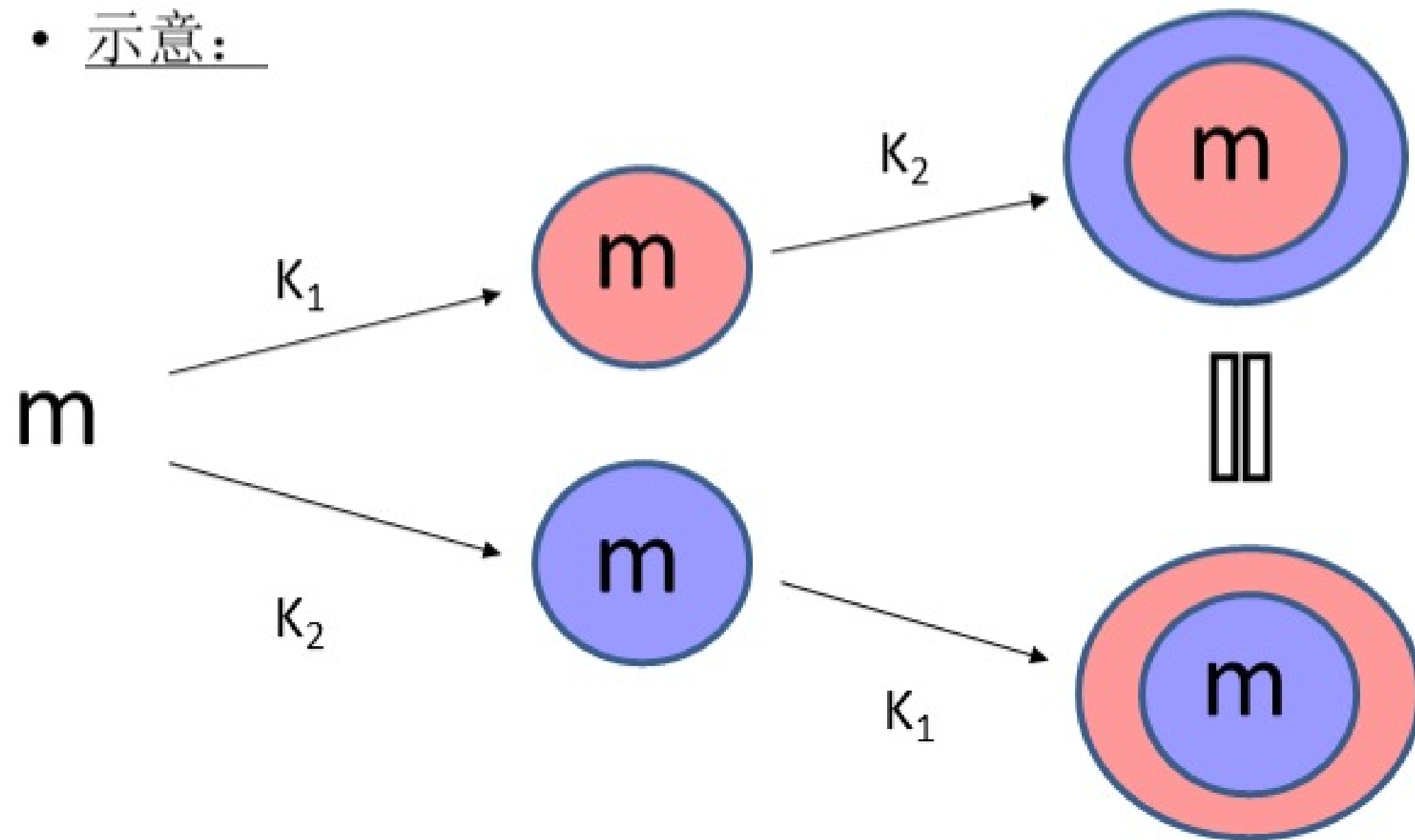
$$C_1 = f_a(f_b(m)) \quad C_2 = f_b(f_a(m)) \quad \rightarrow \quad C_1 = C_2$$

其中 $m \in M$, $a, b \in K$.



3. 基础知识

- 3.可交换加密：不同顺序使用两个密钥加密结果相同
- 基本可交换加密函数：指数幂取模
- 示意：



$$1 \leq M \leq p - 1$$

$$1 \leq C \leq p - 1$$

$$1 \leq K \leq p - 2$$

$$C = M^K (\text{mod } p)$$

$$(M^{K_1})^{K_2} (\text{mod } p) \\ = (M^{K_2})^{K_1} (\text{mod } p)$$

□ 一个常见的可交换加密算法为Pohlig–Hellman加密算法。

4. OPRF定义

- 不经意伪随机函数（Oblivious Pseudorandom Function, OPRF）是一种密码学协议，一方有输入 x ，一方有密钥 key ，OPRF允许输入方得到一个伪随机函数的输出 $F(key, x)$ 。在这个过程中，输入方不知道密钥 key ，另一方也不知道输入 x 。



- 通常来说，OPRF的底层实现一般为不经意传输技术（Oblivious transfer），每个OPRF实例的摊销成本大约需要500比特的通信费用和一些对称密钥操作。

汇报目录

「01」

知识点一：基础知识

「02」

知识点二：朴素哈希PSI

「03」

知识点三：Google PSI

「04」

知识点四：非平衡PSI优化

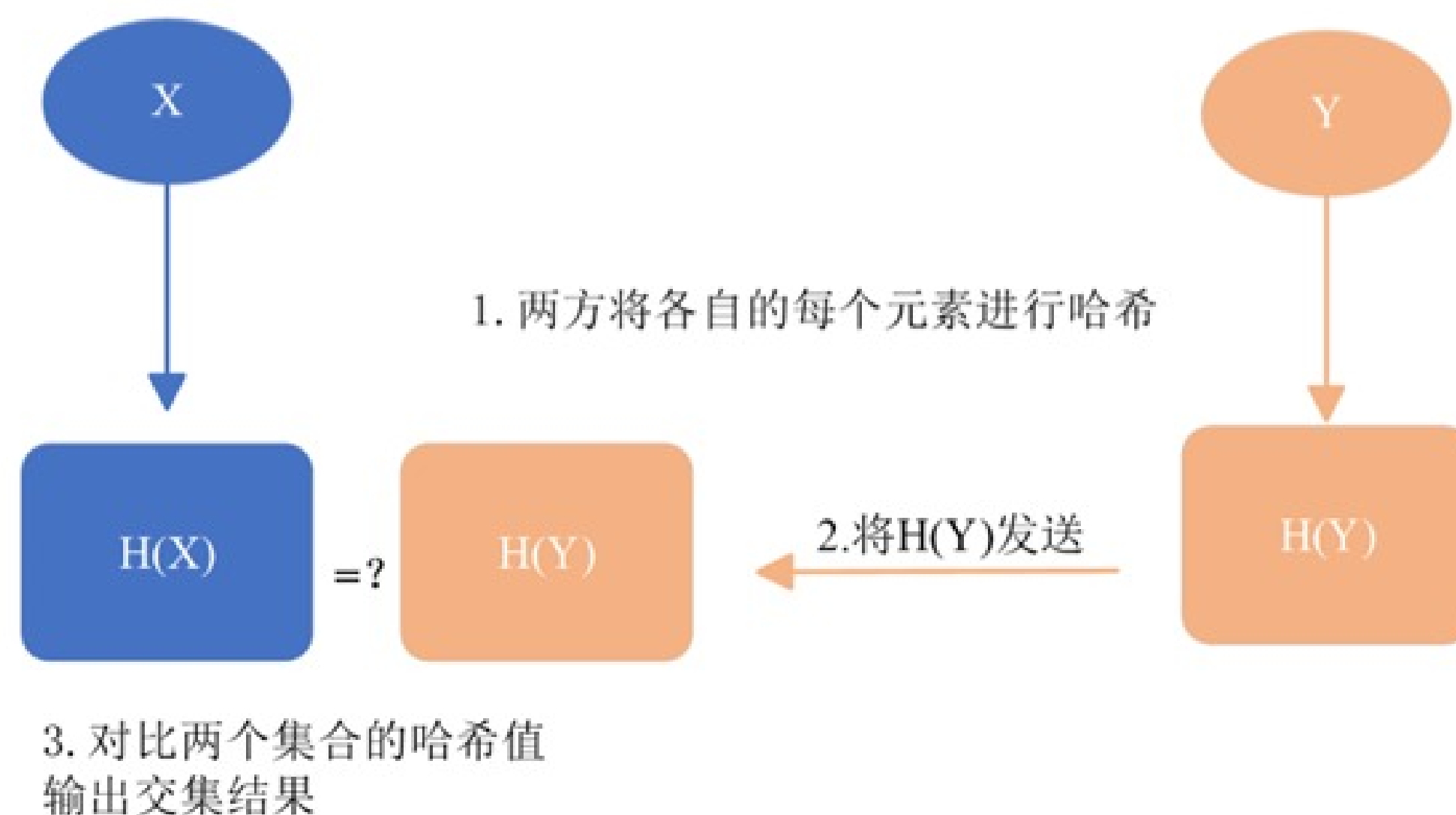
「05」

知识点五：基于OPRF的PSI

1. 算法

当考虑隐私集合求交这个问题时，我们可能会提出下述解决方案：Alice拥有集合X，Bob拥有集合Y。

- 他们分别对X中的元素求哈希，对Y中的元素求哈希。
- Bob随后将哈希值发送给Alice。
- Alice对比两个集合的哈希值，并输出哈希值相等的元素，即交集元素。



2. 安全性

这个协议效率非常高，因为协议只涉及到哈希值的计算。

协议涉及的通信量也很小。

但不幸的是，这个方案是不安全的：因为这个方案会泄露**Bob**输入集合的隐私。

为什么呢？

- ✓ 如果X属于比较小的域，例如X为电话号码，只包含大约10个数字，Alice直接计算上亿个电话号码的哈希值，并将结果与从Bob收到的结果对比。
- ✓ 这样，Alice就可以知道Bob的输入了。这也是此协议被称为朴素哈希的根本原因。
- ✓ 因此，这是一个不安全的**PSI**协议。

汇报目录

「01」

知识点一：基础知识

「02」

知识点二：朴素哈希PSI

「03」

知识点三：Google PSI

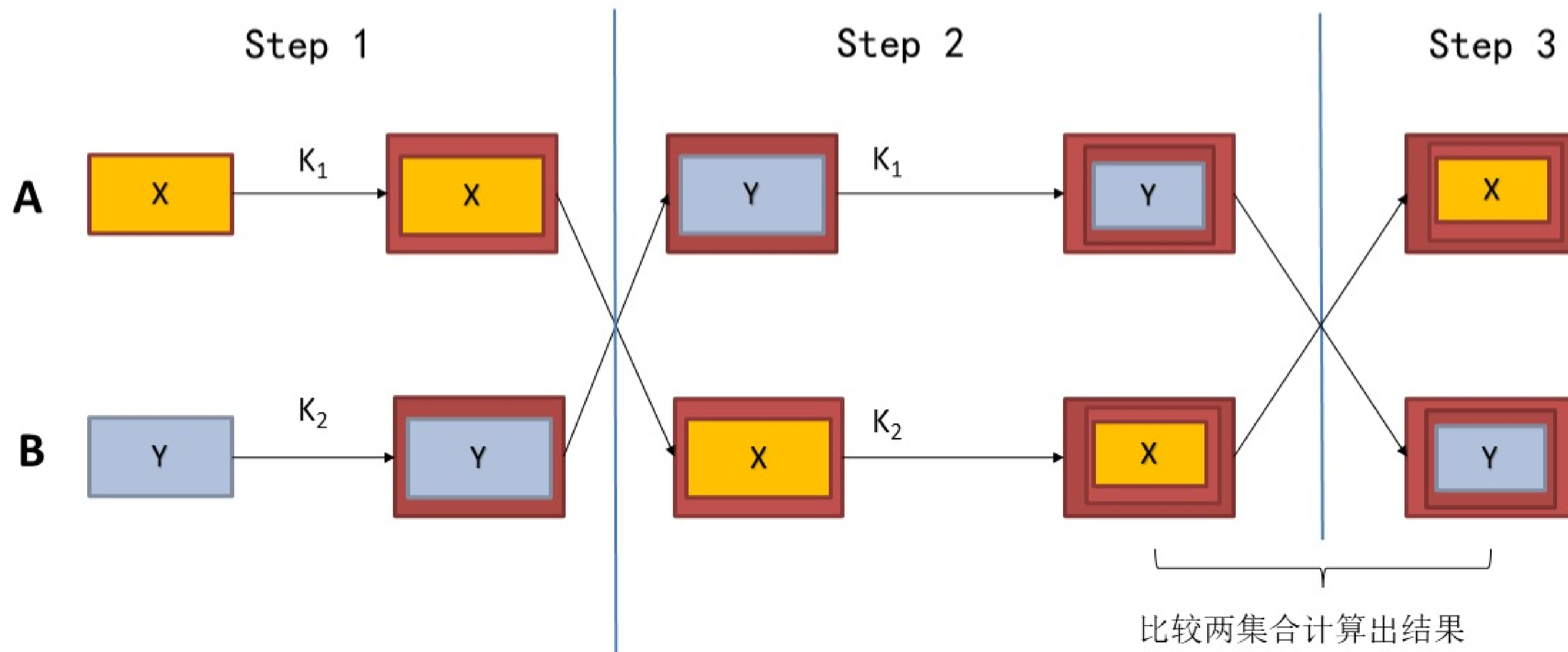
「04」

知识点四：非平衡PSI优化

「05」

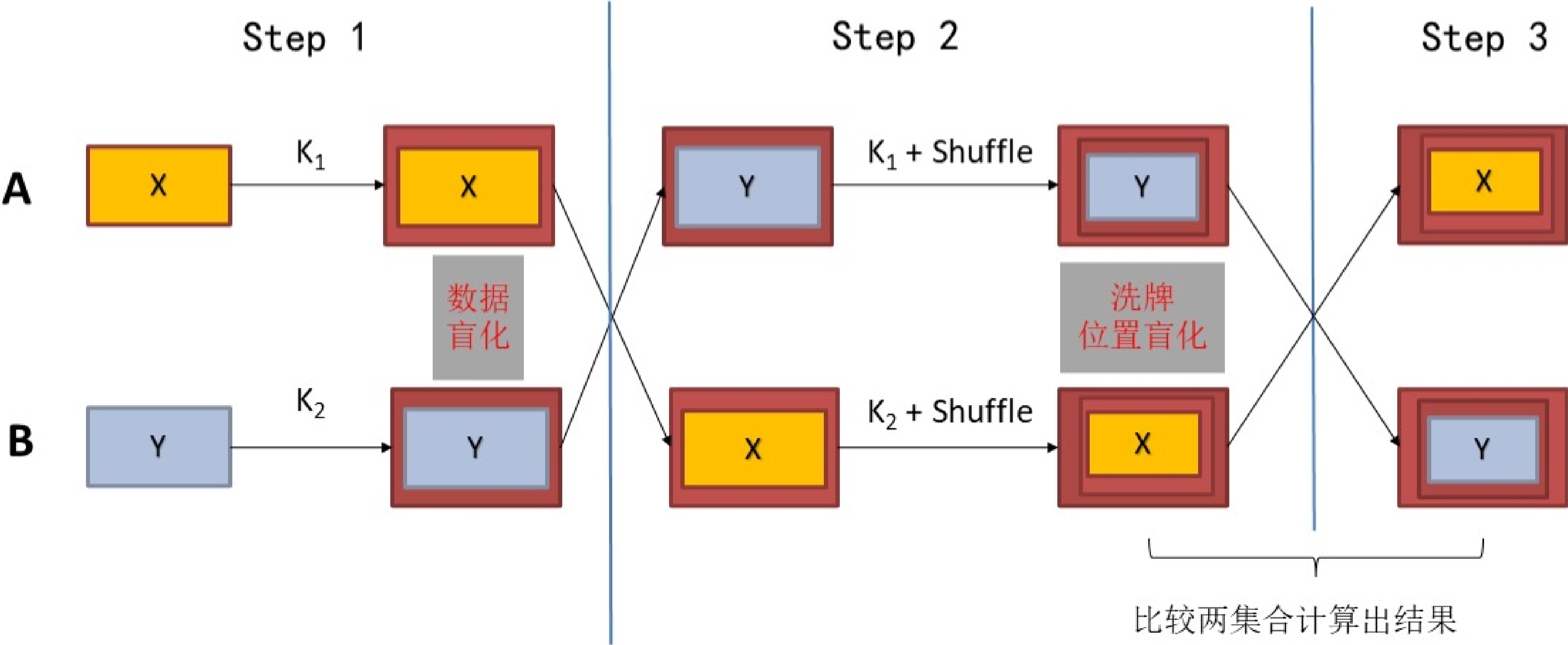
知识点五：基于OPRF的PSI

1. Google的典型PSI协议 (2017年提出)



2.用来构建PSI-CA协议

洗牌操作：让双方只能知道交集个数，不知道交集内容



3. 安全性

□半诚实模型 (Honest but curious) 可证明安全

- 遵循了协议的执行
- 根据预定的程序公平的产生随机数和执行输入与输出
- 保存了协议的中间计算状态
- 半诚实敌手模型在实际的生产生活中普遍存在

□选择明文安全

- 每一次都需要更换新的密钥，即一次一密
- 如果不这么做，在多次PSI计算中，会泄露每次参与的记录数是否相同

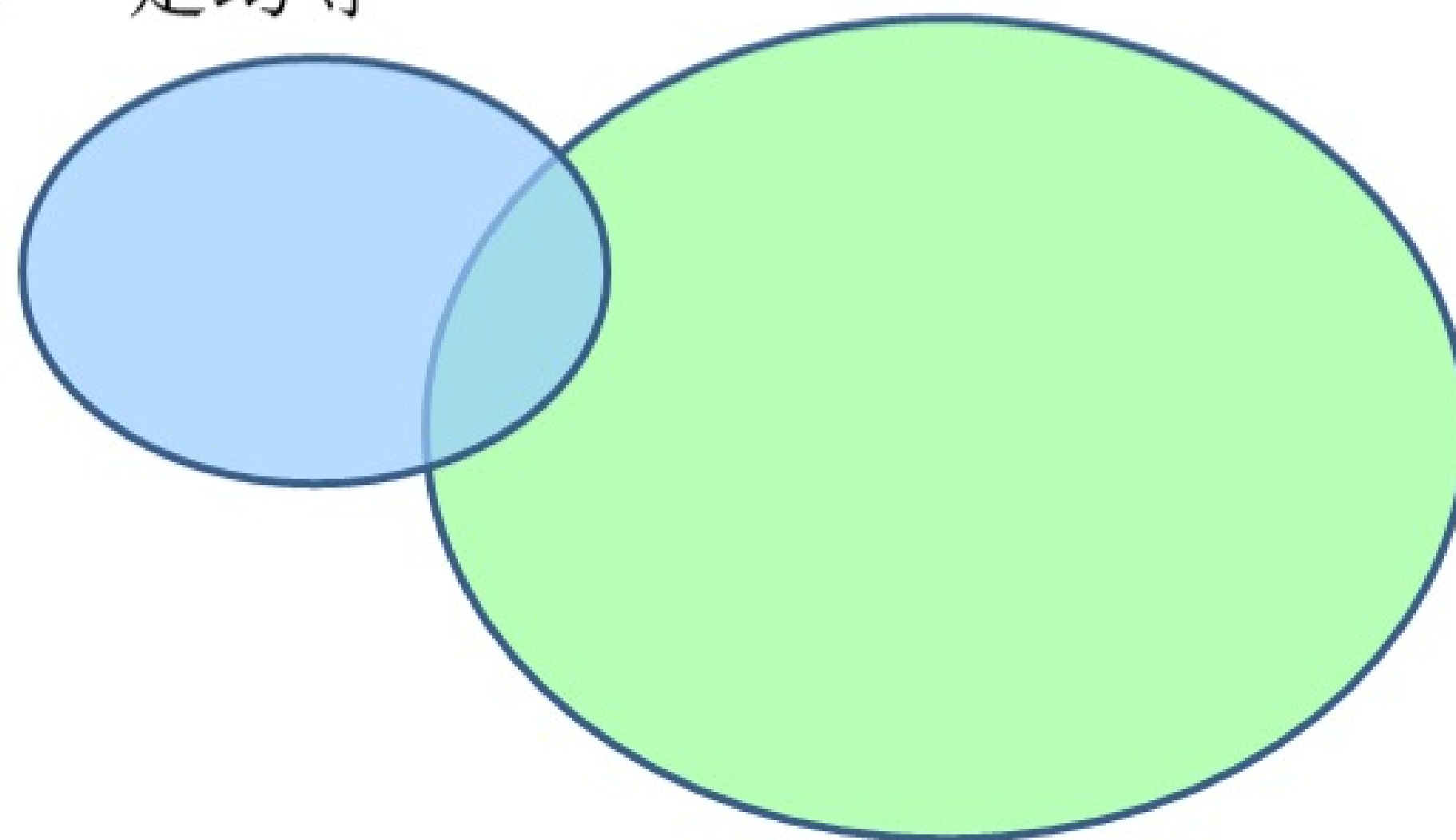
4. 缺点

- 缺点

- 通信量 $2(|A|+|B|)$
- 加解密 $|A|+|B|$
- 现实应用场景两方数据大小和计算能力不一定均等



非平衡集合



汇报目录

「01」

知识点一：基础知识

「02」

知识点二：朴素哈希PSI

「03」

知识点三：Google PSI

「04」

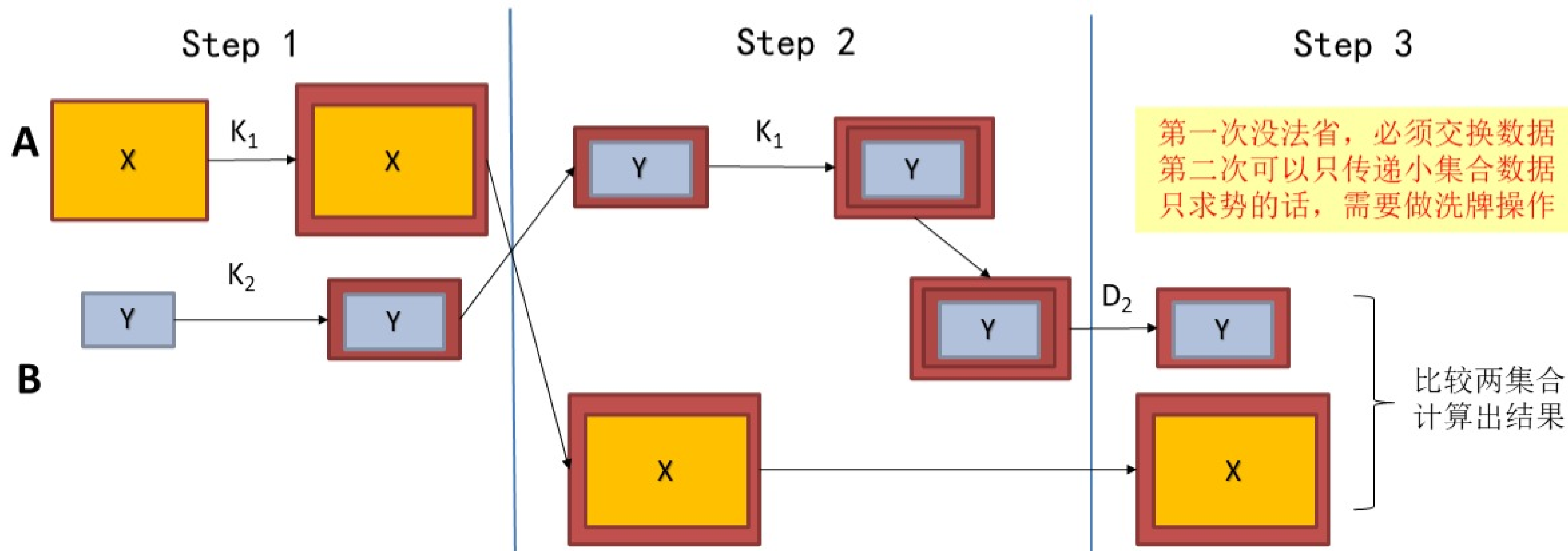
知识点四：非平衡PSI优化

「05」

知识点五：基于OPRF的PSI

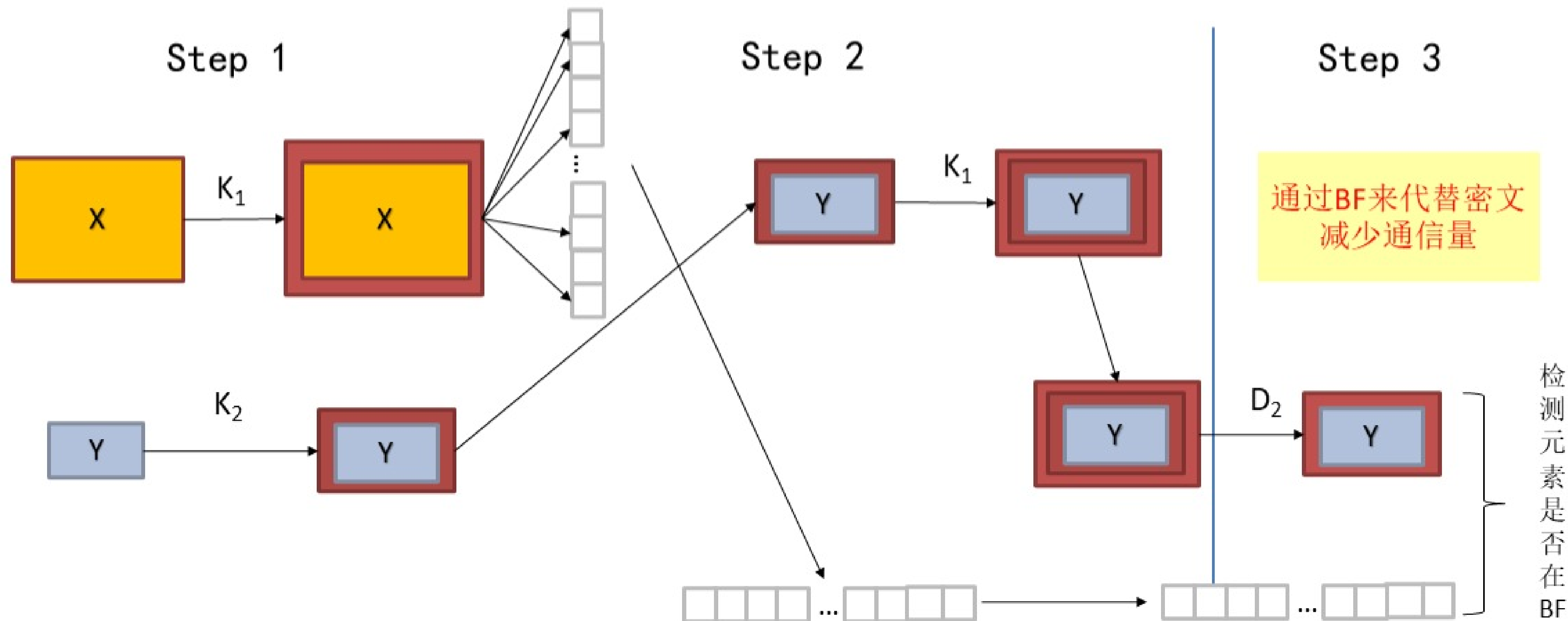
1.低通信量隐私集合交集求势算法

- 非平衡集合场景：减少较大规模的数据的计算与传输；将较大规模的数据的计算与通信转移到较小规模的数据一方。



1.低通信量隐私集合交集求势算法

- 进一步降低通信量：通过发送布隆过滤器取代完整集合降低通信量。



1.低通信量隐私集合交集求势算法

• 典型PSI方案 VS 本方案

➤ **准确性**: 准确 VS 有较小误差

➤ **安全性**:

✓ 半诚实模式(**Honest but curious**) 依旧可证明安全

✓ 新方案在发生法律纠纷的时候**可验证能力减弱**

旧方案, 数据完全交换, 事后验证可以通过交互密钥来完成验证

新方案, 数据不完全交换, 交集通过一方计算后发送给另外一方, 较难验证

• 安全性考虑—如何验证一方有恶意行为

➤ **旧方案**: 均假设集合大小 $|X|$ 与 $|Y|$ 为公开的信息。双方可以通过验证加密过后的集合是否与最初的集合大小相等, 来验证某一方是否在协议执行过程中增加或删除了元素。

➤ **新方案**: 由于布隆过滤器的使用, 接收方无法直接验证接收到的布隆过滤器是发送方根据协议生成的, 或仅使用了发送方集合中的部分元素或向其中插入了本不在发送方集合中的元素。

1.低通信量隐私集合交集求势算法

- 对布隆过滤器进行限制

将一个包含X个元素的集合插入到m个
比特位k个哈希函数的布隆过滤器中，
布隆过滤器中比特位为1的个数期望为

$$m \left(1 - \left(1 - \frac{1}{m} \right)^{kX} \right)$$

限制发送方布隆过滤器中比特位为1的位数等于

$$m \left(1 - \left(1 - \frac{1}{m} \right)^{kX} \right)$$

- 若实际位数大于该值，则将随机位置为0
- 若实际位数小于该值，则将随机位置为1

1.低通信量隐私集合交集求势算法

• 限制引入的误差

添加1 小

向布隆过滤器中删除1的个数不多于

$$\Phi^{-1}(p) \sqrt{m \left(1 - \left(1 - \frac{1}{m}\right)^{k|X|}\right) \left(1 - \frac{1}{m}\right)^{k|X|}}$$

添加的元素占接收方集合的比例为

$$\frac{(1-r)|Y|}{|U| - |X|} \times \frac{\Phi^{-1}(p)r \sqrt{m \left(1 - \left(1 - \frac{1}{m}\right)^{k|X|}\right) \left(1 - \frac{1}{m}\right)^{k|X|}}}{\bar{N} - \Phi^{-1}(p) \sqrt{m \left(1 - \left(1 - \frac{1}{m}\right)^{k|X|}\right) \left(1 - \frac{1}{m}\right)^{k|X|}}}$$

删除1 从集合里删

向布隆过滤器中删除1的个数为

$$\Phi^{-1}(p) \sqrt{m \left(1 - \left(1 - \frac{1}{m}\right)^{k|X|}\right) \left(1 - \frac{1}{m}\right)^{k|X|}}$$

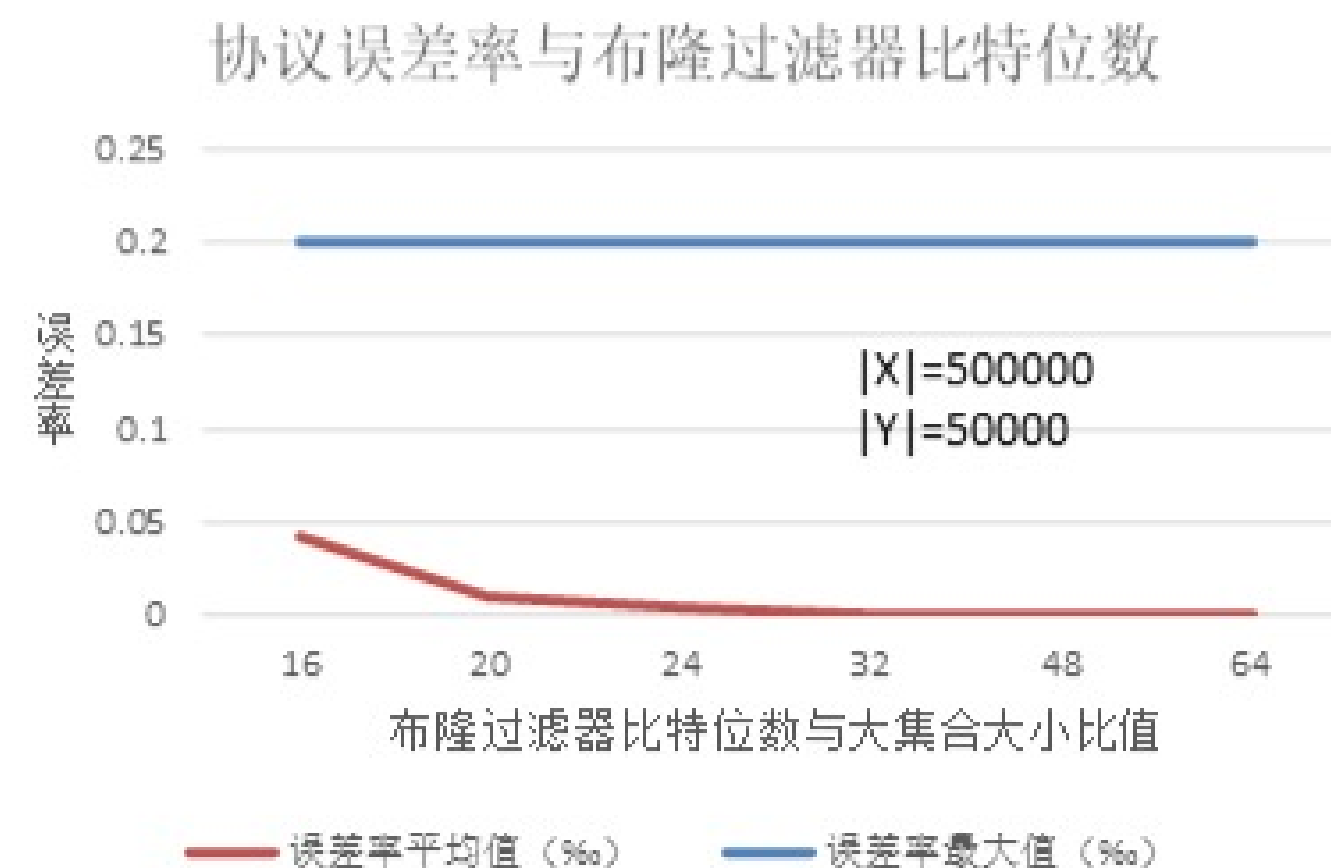
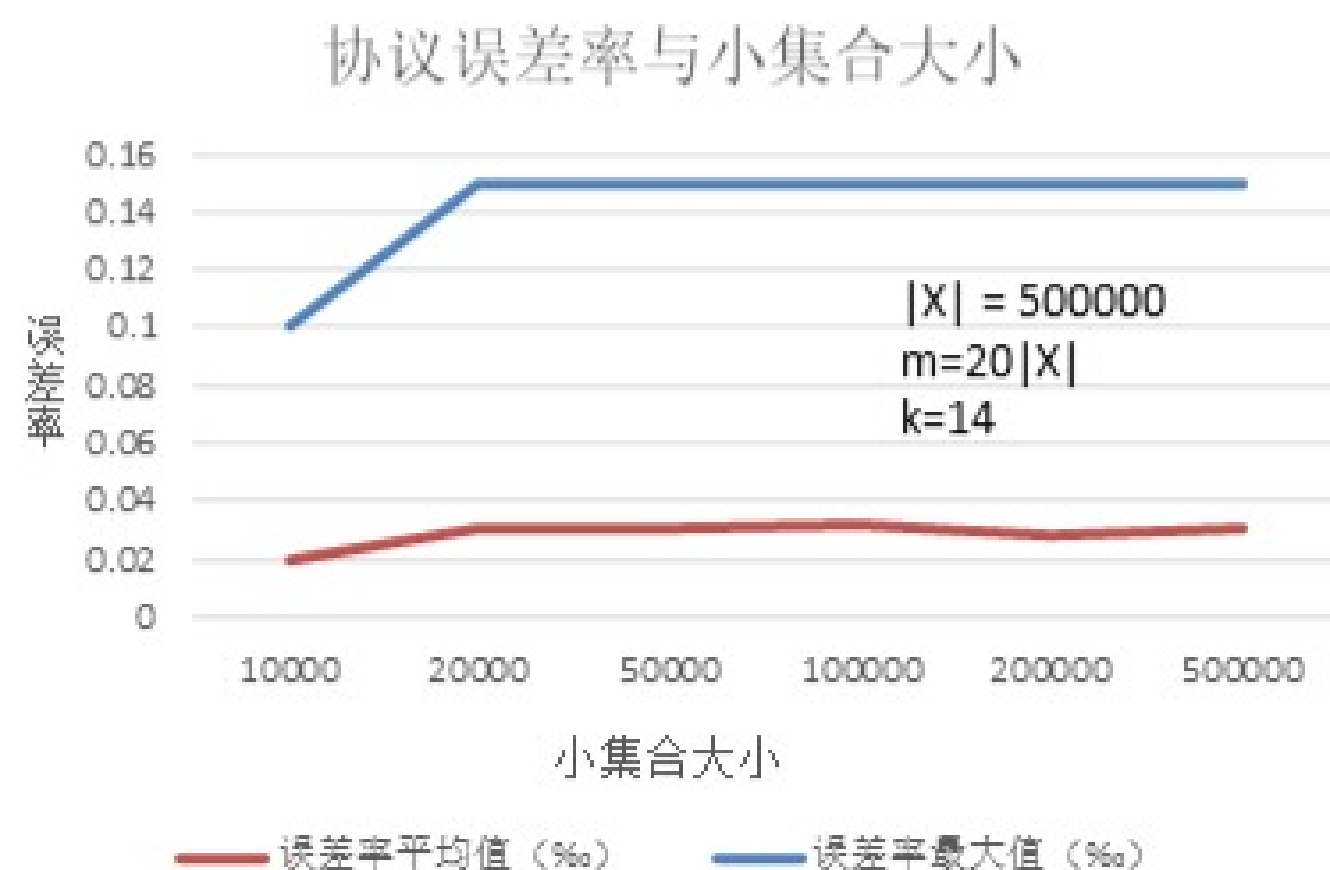
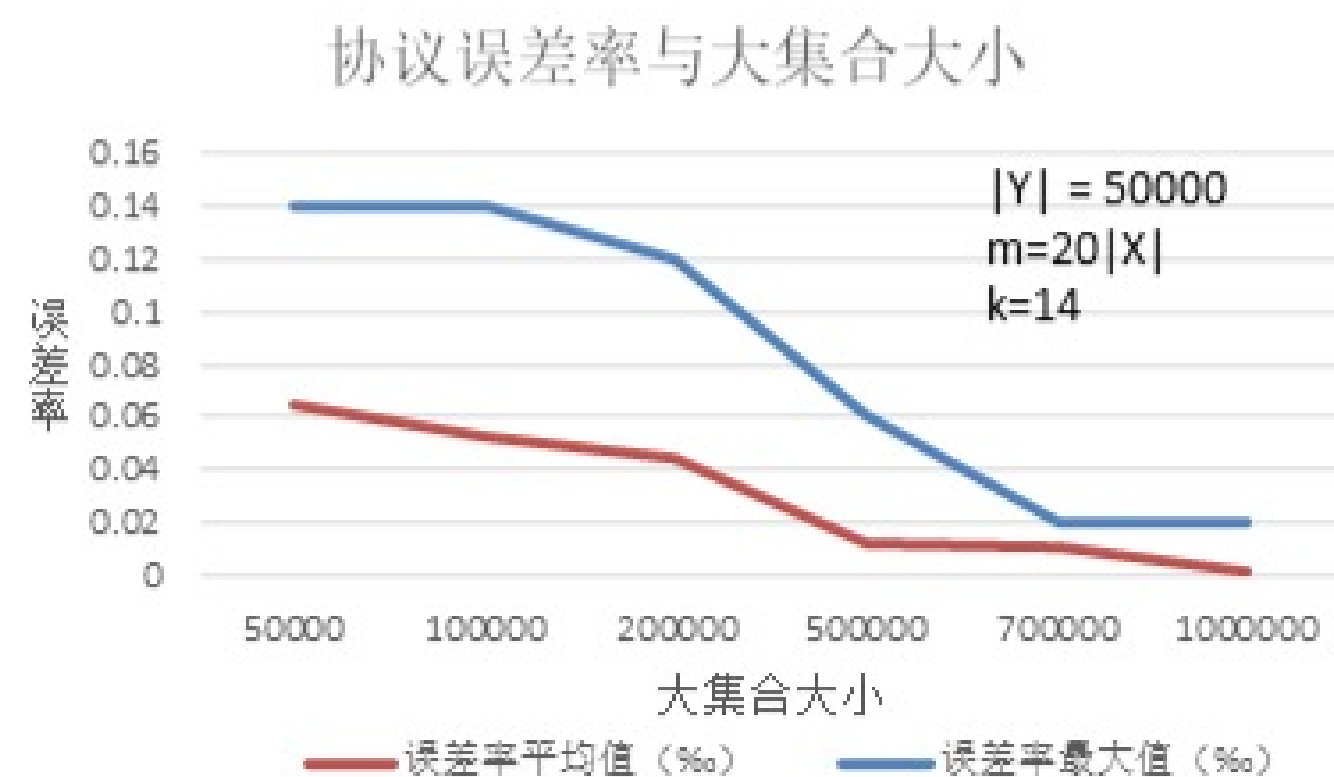
删去的元素占接收方集合的比例为

$$\frac{\Phi^{-1}(p)r \sqrt{m \left(1 - \left(1 - \frac{1}{m}\right)^{k|X|}\right) \left(1 - \frac{1}{m}\right)^{k|X|}}}{\bar{N} + \Phi^{-1}(p) \sqrt{m \left(1 - \left(1 - \frac{1}{m}\right)^{k|X|}\right) \left(1 - \frac{1}{m}\right)^{k|X|}}}$$

在大集合大于500000时（布隆过滤器处理），误差元素占小集合的比例在98%的情况下小于千分之一

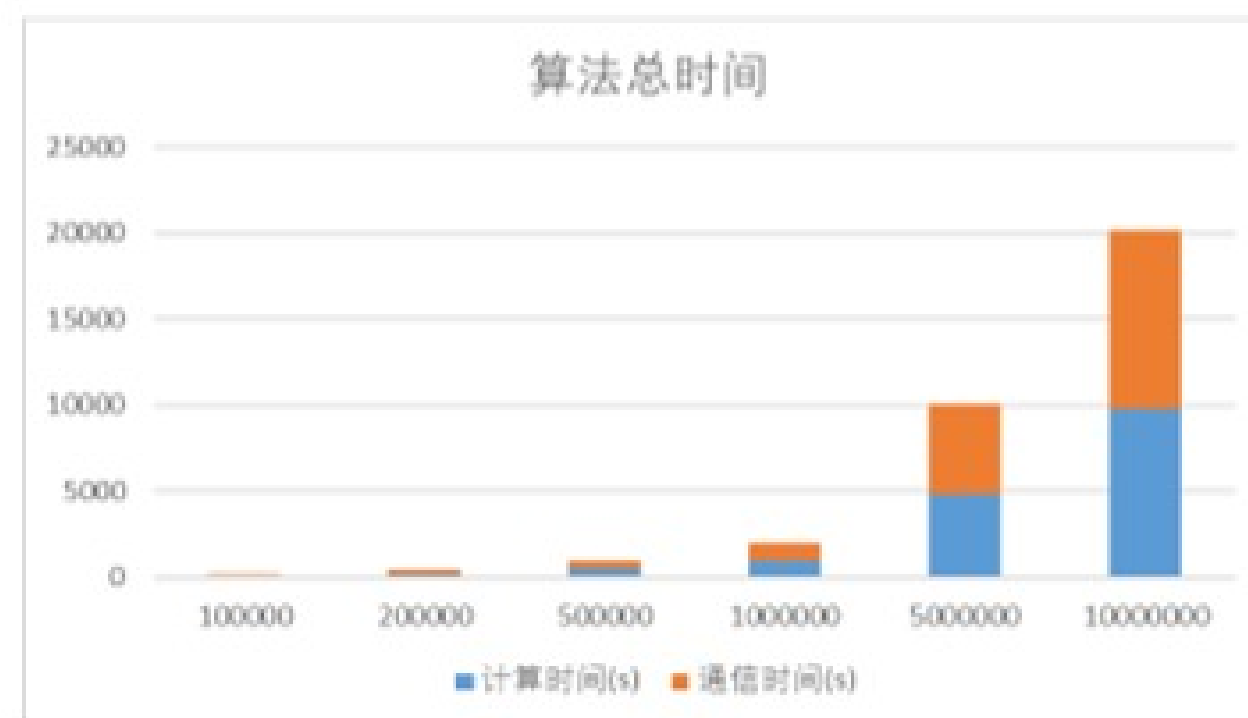
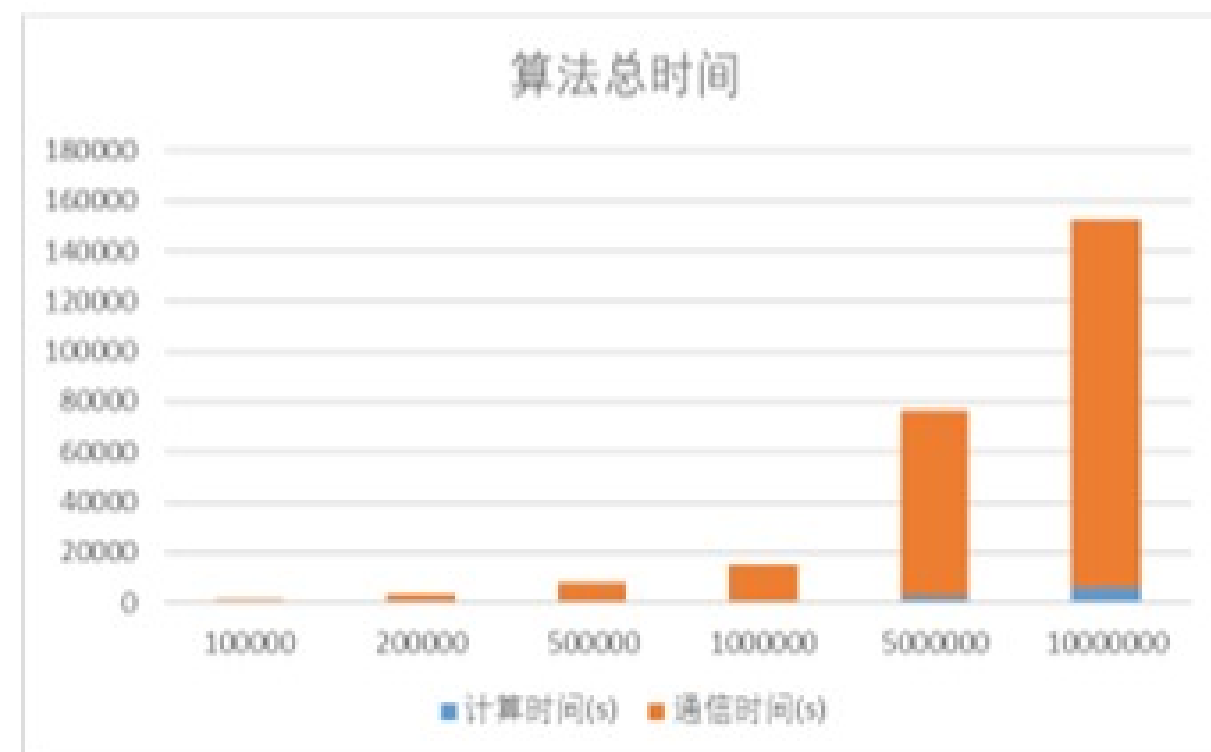
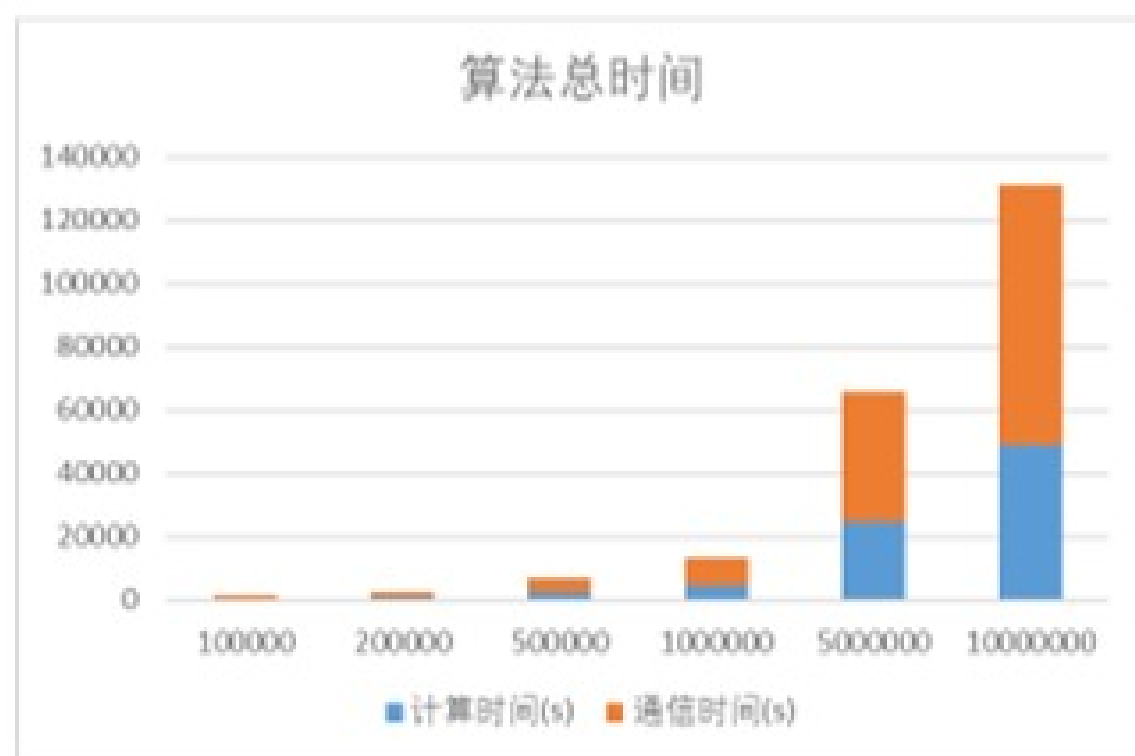
1.低通信量隐私集合交集求势算法

- 误差实验
- 每组数据实验20次，取误差的平均值和最大值；交集以小集合元素50%概率生成
- 记误差率为误差元素个数/小集合元素个数。



1.低通信量隐私集合交集求势算法

- 性能实验 通信时间按10M/S 千万数据大约需花费5600秒



千万数据大约需花费130000秒^[1]

千万数据大约需花费150000秒^[2]

千万数据大约需花费20000秒^[3]

[1] Egert R, Fischlin M, Gens D, et al. Privately computing set-union and set-intersection cardinality via bloom filters[C]. Australasian Conference on Information Security and Privacy. Springer, Cham, 2015: 413-430.

[2] Debnath S K, Dutta R. Secure and efficient private set intersection cardinality using bloom filter[C]. International Conference on Information Security. Springer, Cham, 2015: 209-226.

[3] Ion M, Kreuter B, Nergiz E, et al. Private Intersection-Sum Protocol with Applications to Attributing Aggregate Ad Conversions[J]. IACR Cryptology ePrint Archive, 2017, 2017: 738.

汇报目录

「01」

知识点一：基础知识

「02」

知识点二：朴素哈希PSI

「03」

知识点三：Google PSI

「04」

知识点四：非平衡PSI优化

「05」

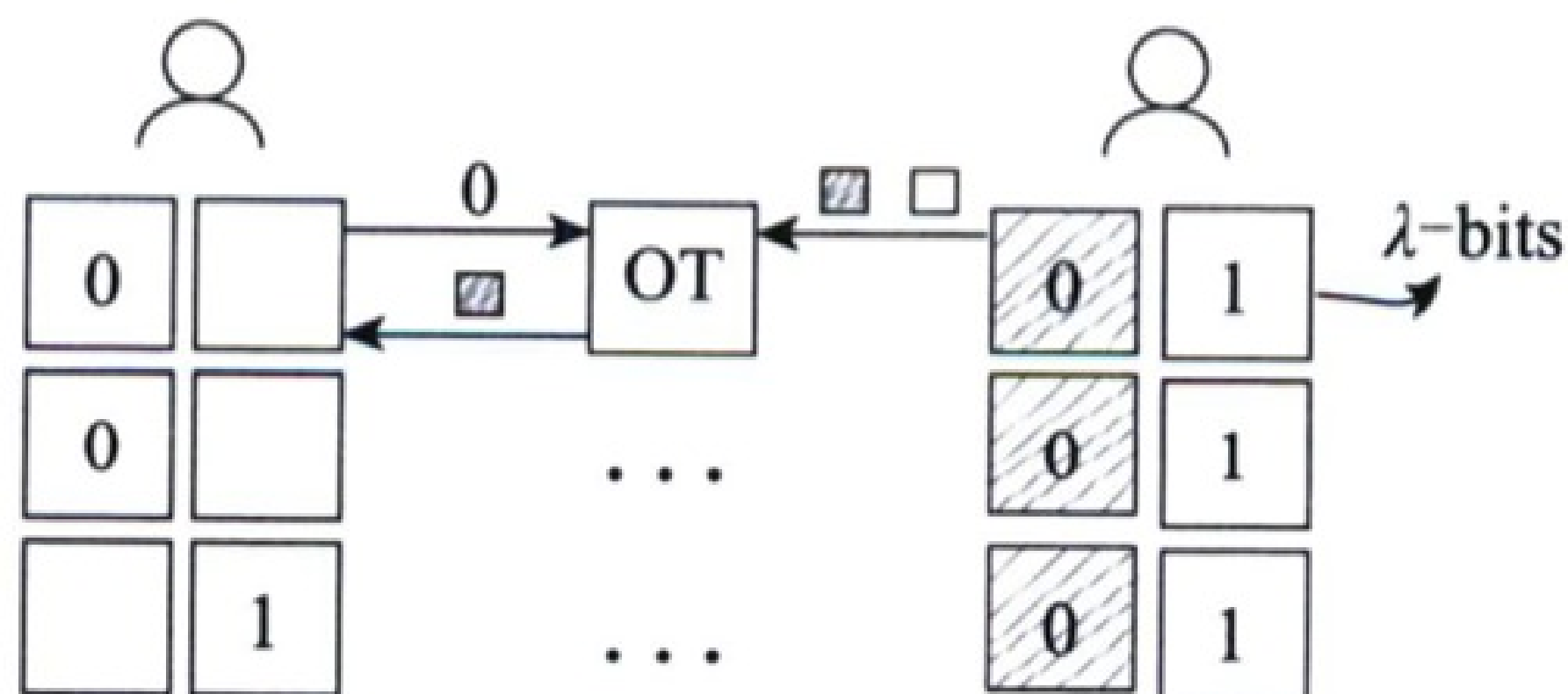
知识点五：基于OPRF的PSI

1. 从隐私比较到OPRF

- 我们假设有两个数据方 (P_1 和 P_2)。 P_1 拥有字符串 $x=001$ ，而 P_2 拥有字符串 $y=011$ ，他们想在不泄露各自输入的前提下，比较 x 是否等于 y 。
- 基于OT的两方字符串比较算法如图所示：

例子： $x=001, y=011$ ，比较： $x=y?$

左侧甲方先通过OT，获得自己拥有的0或者1



右侧乙方执行发送数据到甲方，由甲方比对

- 乙发送 λ -bits $0 \oplus 1 \oplus 1$ 给甲
- 甲计算 $0 \oplus 0 \oplus 1$ 后与乙发送的数据进行比较

1. 从隐私比较到OPRF

可以分为以下几个核心步骤：

(1) 其中一方（图5.11中是右侧 P_2 ）生成 $2l$ 长度为 λ 比特串，即 l 个字符串对，每个字符串对包括两个 λ 比特串，分别对应0和1两位。这里的 l 指的是双方待比较的字符串的长度，在图5.11的例子中， $l=3$ 。 λ 是指一个超参数，它的长度越大，该算法的安全性及正确性也就越高。

(2) P_1 对待比较字符串（ x ）的每一位， P_1 和 P_2 使用OT，使得 P_1 能够获取 P_2 每个字符串对中的一个 λ 比特串。具体而言， P_1 作为OT的接收者， P_2 作为OT的发送者； P_2 拥有两个长度为 λ 的比特串， P_1 输入0或1，返回 P_2 该位字符（0或1）所对应的比特串。在这个过程中、由于采用OT， P_1 不知道 P_2 另外一个比特串是什么，同时， P_2 不知道 P_1 请求的是哪一个比特串。重复待比较字符串的每一位， P_1 便得到了 l 个长度为 λ 的比特串。然后 P_1 对这 l 个比特串做异或，得到一个字符串（记为 S_1 ）。

1. 从隐私比较到OPRF

(3) P_2 对其待比较字符串 (y) 的每一位, 选择每个字符串对相应的比特串, 这样 P_2 也得到了1个长度为2的比特串。然后 P_2 对这1个比特串做异或, 得到一个字符串 (记为 S_2), 并将 S_2 发送给 P_1 。

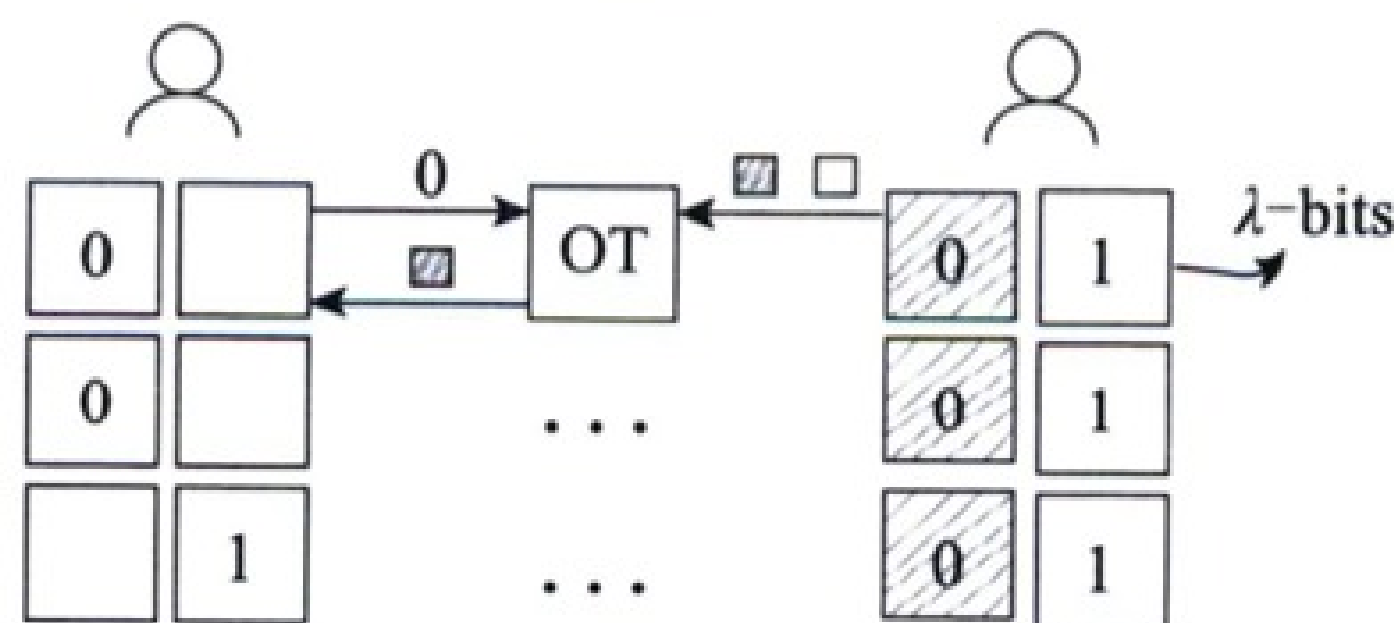
(4) P_1 比较 S_1 和 S_2 , 如果 S_1 和 S_2 相等, 那么就说明 P_1 和 P_2 的两个原待比较字符串也相等, 反之说明它们不相等。

1. 从隐私比较到OPRF

- 观察隐私比较，我们可以发现，发送方 P_2 方持有一组二进制串，我们可以将这些二进制串整体当作一个随机种子 k ，由 P_2 方持有。从 P_1 方的角度来看，隐私比较的过程，就是 P_1 方输入数据 x ，得到一个随机二进制串 $F(k, x)$ ，这个二进制串由 P_2 方持有的随机种子 k 与输入 x 来决定，同时 P_2 方无法得知 P_1 方的输入 x 。
- 这一过程，可以看作使用不经意伪随机函数，来构建了一个隐私比较算法。接下来，看看如何使用不经意伪随机函数，来构建隐私集合求交。

第一步：P1方得到001对应的密文，相当于执行了 $F(k, x)$

例子： $x=001, y=011$, 比较： $x=y?$



P2方的 $y=011$,理解为 k

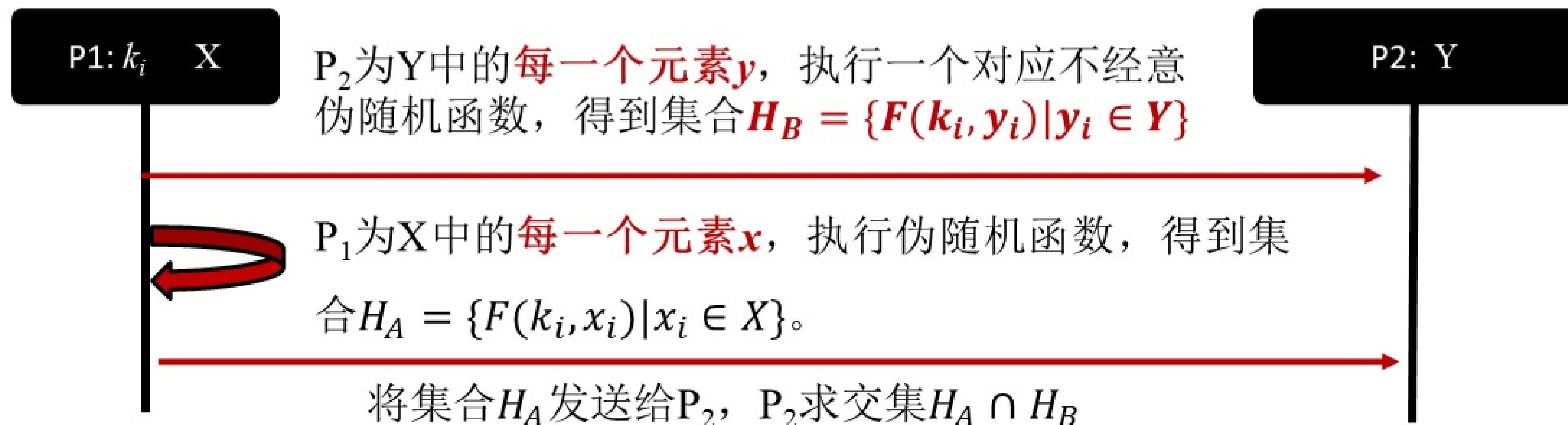
- 乙发送 λ -bits $\boxed{0} \oplus \boxed{1} \oplus \boxed{1}$ 给甲
- 甲计算 $\boxed{0} \oplus \boxed{0} \oplus \boxed{1}$ 后与乙发送的数据进行比较

2. 基于OPRF的PSI

(1) 基础构造

假设 P_1 持有一组输入 X ， P_2 持有一组输入 Y ， $|X|=|Y|=n$ 。通过不经意伪随机函数，我们可以构造出一个非常朴素的隐私集合求交算法：

- ① P_1 构造 n 个不经意伪随机函数的种子 k_i ， $i \in \{0, 1, 2, \dots, n-1\}$ 。
- ② P_2 为 Y 中的每一个元素 y ，执行一个对应不经意伪随机函数，得到集合 $H_B = \{F(k_i, y_i) | y_i \in Y\}$ 。
- ③ P_1 为 X 中的每一个元素 x ，执行伪随机函数，得到集合 $H_A = \{F(k_i, x_i) | x_i \in X\}$ 。
- ④ P_1 将集合 H_A 发送给 P_2 ， P_2 求交集 $H_A \cap H_B$ ，再将交集映射回 Y ，即可得到 X 与 Y 的交集。



2. 基于OPRF的PSI

(1) 基础构造

- ▣ 简单来讲，就是 P_2 将每一个 Y 中的每一个元素，都与 P_1 的 X 中的每一个元素通过不经意伪随机函数进行隐私比较，进而得到 X 与 Y 的交集。
- ▣ 假设双方各有 n 个元素，那么求交集过程中，需要比较的次数为 $O(n^2)$ ：因为 H_A 的大小是 $O(n^2)$ 。对不同的 y 对应的密钥 k_i ，要对 X 中每个元素都提供密文，才能保证整个比较过程是正确的。

3. 基于布谷哈希的优化方案

布谷哈希

- 该协议要用到包含3个哈希函数的布谷鸟哈希（Cuckoo Hashing）（Pagh and Rodler, 2004）。我们现在简要介绍布谷鸟哈希的基本原理。
- 两个关键点：
 - 布谷鸟妈妈从不筑巢，它将自己的鸟蛋生在其他鸟类的巢穴里，要别的鸟给它孵蛋
 - 新生的布谷鸟会本能地将巢穴里的其他蛋踢开（kick out），推出鸟巢，以确保自己在鸟巢里可以独享宠爱
- 为应用布谷鸟哈希将 n 个元素分配到 b 个箱子中，首先选择 3个随机哈希函数 $h_1, h_2, h_3: \{0,1\}^* \rightarrow [b]$ ，并初始化 b 个空箱子 $B[1, \dots, b]$ 。

3. 基于布谷哈希的优化方案

布谷哈希

- 为计算元素 x 的哈希值，首先检查 $B[h_1(x)], B[h_2(x)], B[h_3(x)]$ 这三个箱子中是否有一个是空箱子。如果至少有一个箱子是空的，则将 x 放置在其中一个空箱子内，并终止算法。否则，随机选择 $i \in \{1, 2, 3\}$ ，将 $B[h_i(x)]$ 中的当前元素驱逐出箱子，将 x 放置在此箱子中，并向其他箱子迭代插入被驱逐的元素。
- 如果经过一定次数的迭代之后算法仍未终止，则将最后被驱逐出的元素放置在一个名为暂存区（Stash）的特殊箱子中。
- 布谷鸟哈希的一个重要性质：对于某个元素 x ，其所在桶的位置必然是 $h_1(x), h_2(x), h_3(x)$ 中的一个，或者暂存区的一个。

3. 基于布谷哈希的优化方案

应用布谷鸟哈希的PSI

- 首先，两个参与方为3-布谷鸟哈希选择3个随机哈希函数 h_1, h_2, h_3 。假设 P_1 的输入集合为 X ， P_2 的输入集合为 Y ，且满足 $|X|=|Y|=n$ 。
- P_2 应用布谷鸟哈希将集合 Y 中的元素放置在 $1.2n$ 个箱子和大小为 s 的暂存区中。
- 此时， P_2 的每个箱子中最多含有一个元素，暂存区中最多含有 s 个元素。
- P_2 用虚拟元素填充箱子和暂存区，使每个箱子均包含一个元素，暂存区中包含 s 个元素。

3. 基于布谷哈希的优化方案

应用布谷鸟哈希的PSI

- ▣ 两个参与方随后执行 $1.2n + s$ 个OPRF协议， P_2 作为OPRF协议的接收方，分别将 $1.2n + s$ 个元素作为OPRF的输入。
- ▣ 令 $F(k_i, \cdot)$ 表示第 i 个OPRF协议所对应的PRF。
 - 如果 P_2 通过布谷鸟哈希将元素 y 放在第 i 个箱子中，则 P_2 得到 $F(k_i, y)$;
 - 如果 P_2 将元素 y 放在暂存区中，则 P_2 得到 $F(k_{1.2n+j}, y)$ 。

3. 基于布谷哈希的优化方案

- 另一方面， P_1 可以对任意 i 计算 $F(k_i, \cdot)$ 。因此， P_1 计算得到下述两个候选 PRF 的输出集合：

$$H = \{ F(k_{h_i(x)}, x) \mid x \in X \text{ 且 } i \in \{1, 2, 3\} \}$$

$$S = \{ F(k_{1..2n+j}, x) \mid x \in X \text{ 且 } j \in \{1, \dots, s\} \}$$

- P_1 随机打乱集合 H 和 S 中元素的位置，并将 H 和 S 发送给 P_2 。
- P_2 可按下述方法计算得到 X 和 Y 的交集：
 - 如果 P_2 有一个被映射到暂存区中的元素 y ，则 P_2 验证 S 中是否含有 y 所对应的 OPRF 输出。
 - 如果 P_2 有一个被映射到哈希箱子中的元素 y ，则 P_2 验证 H 中是否含有 y 所对应的 OPRF 输出。

3. 基于布谷哈希的优化方案

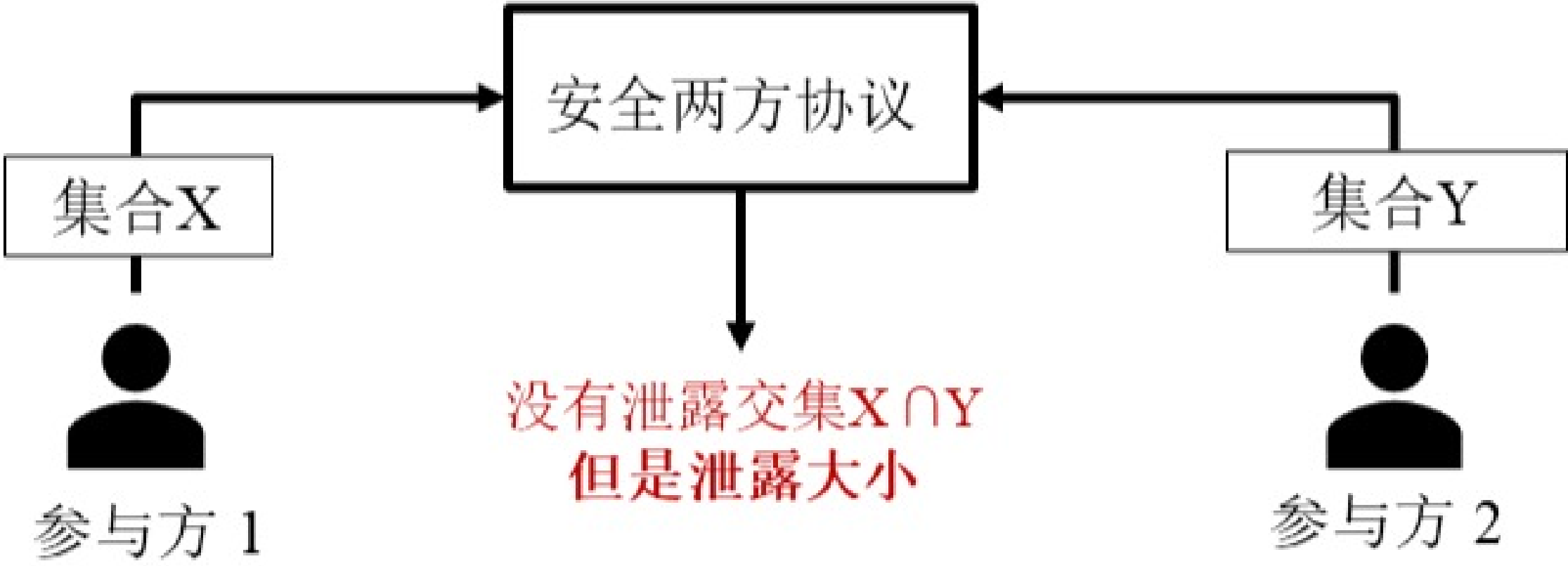
- 通过计算，我们可以发现，集合 H 的大小为 $3n$ ，集合 S 的大小为 s ， s 是一个常数，因此 P_1 需要传输的数据量为 $(s+3)*n$ ，是 $O(n)$ 的。通过结合布谷鸟哈希，我们减少了协议所需要传输的数据量，加快了协议的执行速度。
- 直观上看，此协议可以抵御半诚实 P_2 的攻击。这是因为元素 $x \in X \setminus Y$ 所对应的 PRF 输出 $F(k_i, y)$ 满足伪随机性。类似地，如果密钥具有关联性，但 PRF 的输出仍然满足伪随机性，则用 OPRF 实现关联密钥 PRF 也可以保证方案的安全性。
- 只要 PRF 的输出不发生碰撞（即对于 $x \neq x', F(k_i, x) = F(k_i, x')$ ），此协议的计算结果就是正确的。必须谨慎设置协议的参数，避免 PRF 的输出发生碰撞。
- 核心：通过布谷鸟哈希，将比较的范围缩小了。

汇报目录



密文交集运算成员推理攻击

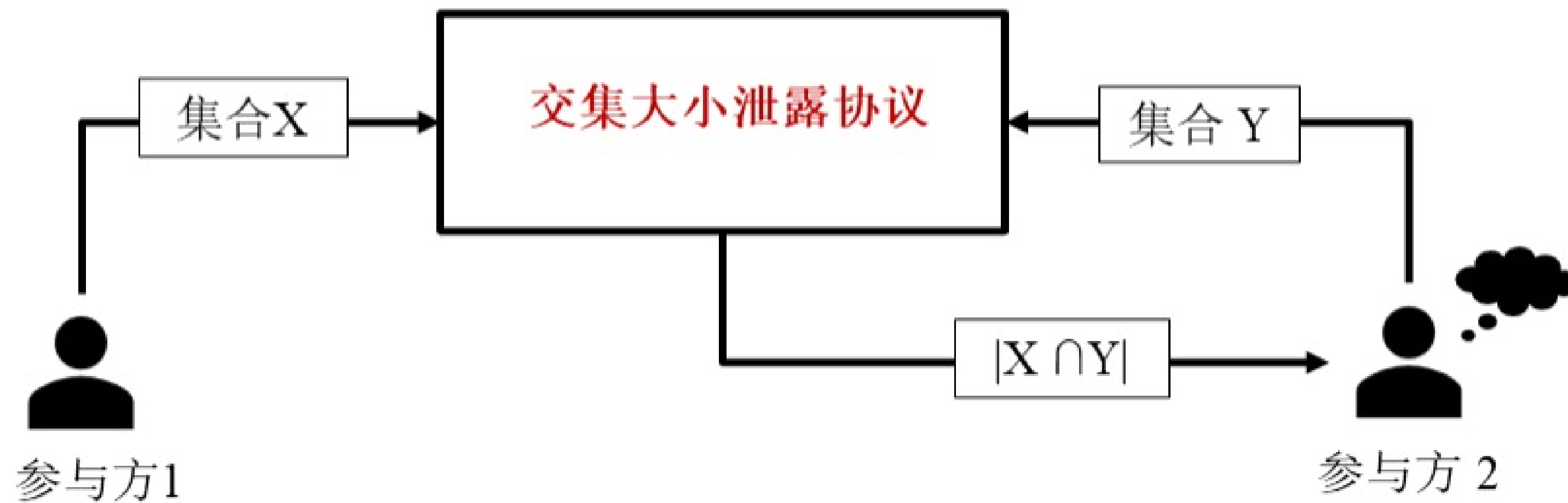
1. 应用情况



大小泄露是公认事实
不允许泄露交集是前提
有什么问题吗?

协议	场景	系统	Developer
PSI-CA及其变种	COVID-19 接触者追踪	Epione Catalic COVID-19冠状病毒病警告	Researchers Researchers Openmined
PSI-SUM	广告转化率	Private Join and Compute	Google
Private-ID	广告转化率	Private Lift	Facebook
数据对齐	纵向联邦机器学习	FATE等	Tencent

2. 集合成员推理问题



- 集合成员推理:

拥有许多目标元素的一方可以确定这些元素是否在对方集合中。

2. 集合成员推理问题

威胁模型

攻击者的能力

- 是交集大小泄露协议的一部分
- 可以选择输入
- 可以得知交集大小
- 与相同受害者执行多次交集协议

攻击者的背景知识

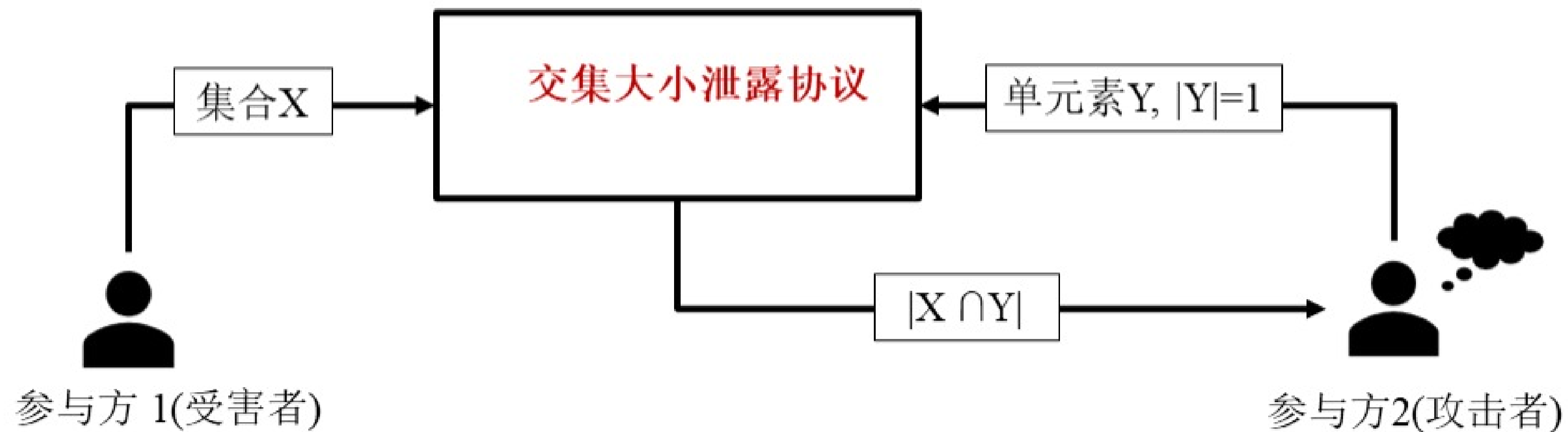
- **baseline**攻击者：不掌握任何信息，只有元素ID。较强假设
- **Feature-aware**攻击者：掌握元素ID及其特征。现实攻击者

攻击者的目标

它的集合元素是否是受害者集合的成员

2. 集合成员推理问题

Toy Attack



攻击者可以使用交集大小来测试某些元素是否在交集中。

$|X \cap Y| = 1$: 在; 否则, 不在。

每次1个元素, 攻击不符合现实情况, 容易被识别

2. 集合成员推理问题

Baseline Attack

- 攻击者: 采用一个**binary-search-like** 策略

1. 构建一个 **binary tree**

- (i) 每个节点存储一个非空集合;
- (ii) 根节点存储所有目标元素组成的集合;
- (iii) 存储在非叶子节点的集合会被随机均匀地划分为两个非空不相交的子集, 分别存储在非叶子节点的两个子节点中。

2. 运行深度优先遍历

对于排队的子树:

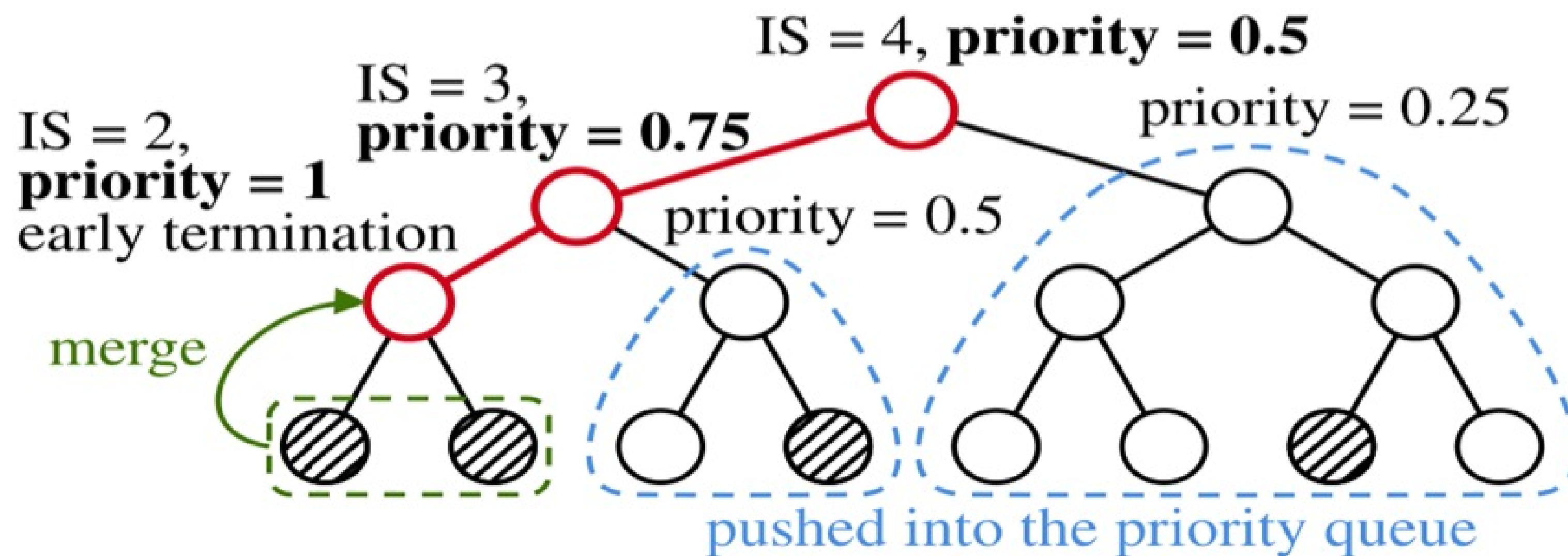
- (i) 从队列中选择一个子树;
- (ii) 使用子树存储的集合与受害者执行一次协议, 得到交集大小;
- (iii) 当交集大小与集合大小相等的时候, 结束遍历。

2. 集合成员推理问题

贪心DFS: 一个例子

IS: 交集大小 集合元素个数: 8个

目标: 交集元素&非交集元素



深度优先遍历GFS: 会首先确定相同深度的起始节点 (比如按编号)

贪心GFS: 根据最优交集大小的集合来调整相同深度的子树及路径

每一层都可以添加一些确定不在集合里的元素, 确保集合大小相同。

2. 集合成员推理问题

Feature-aware Attack

- 遵循与Baseline attack相同的binary-search-like策略
- 但是构建一个不同的**binary tree**

事实

受害者的集合中通常包含具有特定特征的元素，比如新冠病人通常是具有感冒、咳嗽特征的人，我们称之为Set bias。

攻击

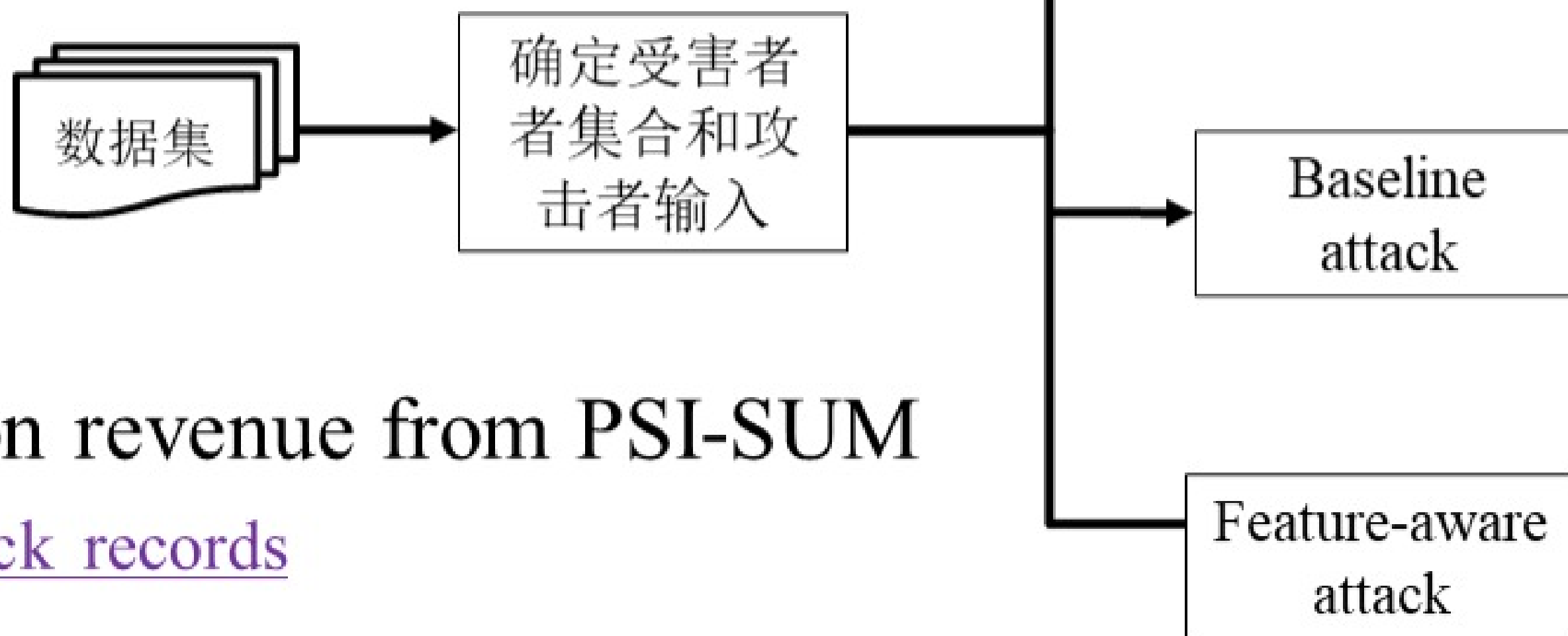
- 攻击者使用已知特征、使用**k-means**将存储在父节点中的集合分层划分为两个子集。
- 与baseline Attack相比，由此产生的二叉树更容易提前终止。

3. 实验评估

1. COVID-19 contact tracing from PSI-CA

- [COVID-19 dataset of tested individuals in Israel](#)

数据集具有时序特征：
解决了我们现实世界动态
变化数据集的问题



2. Measurement of ad conversion revenue from PSI-SUM

- [Taobao's dataset of ad display/click records](#)

3. Measurement of ad conversion lift from Private-ID

- [Tencent's dataset of ad display records](#)

3. 实验评估

主要的评估指标

- **# total**: 确定的集合成员的目标元素的总数，包括交集元素和非交集元素。
- **# members**: 被确定为受害者集合成员的目标元素的数量。
- **# TP , # FP**: 真阳性和假阳性。数据集有变化，可能过去得到的交集大小里包括的元素在后面被调出。真阳性表示确定为交集元素的确实是在交集的比率；假阳性表示确定为交集元素的并非是交集元素的比率。
- **# TN , and # FN**: 真阴性和假阴性。真阴性表示确定为不在交集的元素的确实不在交集的比率；假阴性表示确定为不在交集的元素却在交集的比率。

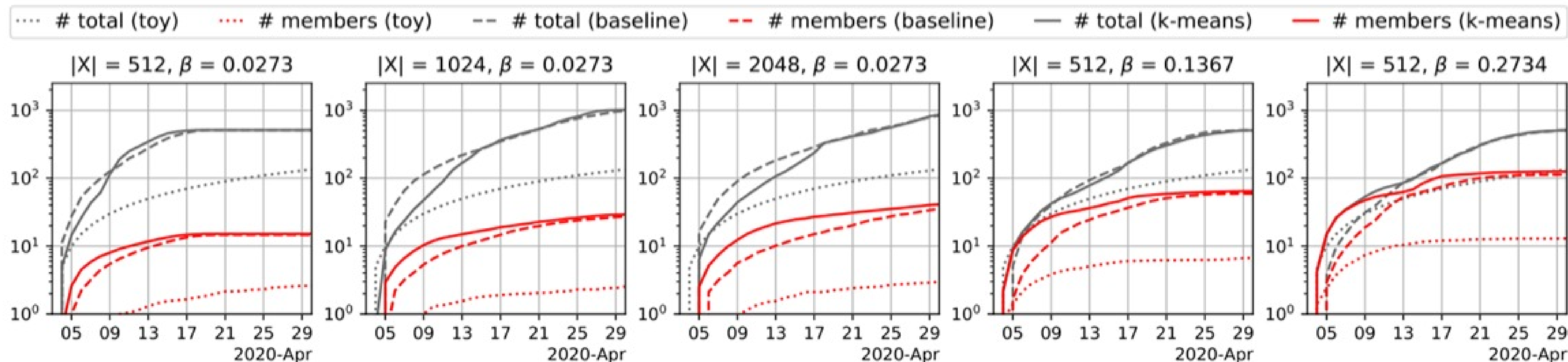
3. 实验评估

Scenario 1: COVID-19 contact tracing

Feature name(MI value –互信息)

fever(0.0168), cough(0.0099), gender(0.0004)

B 表示交集元素占比，假定一天实验可以执行5次协议。横轴：时间；纵轴：数值



- Set membership leakage in **COVID-19 contact tracing**, measured in # total and # members.

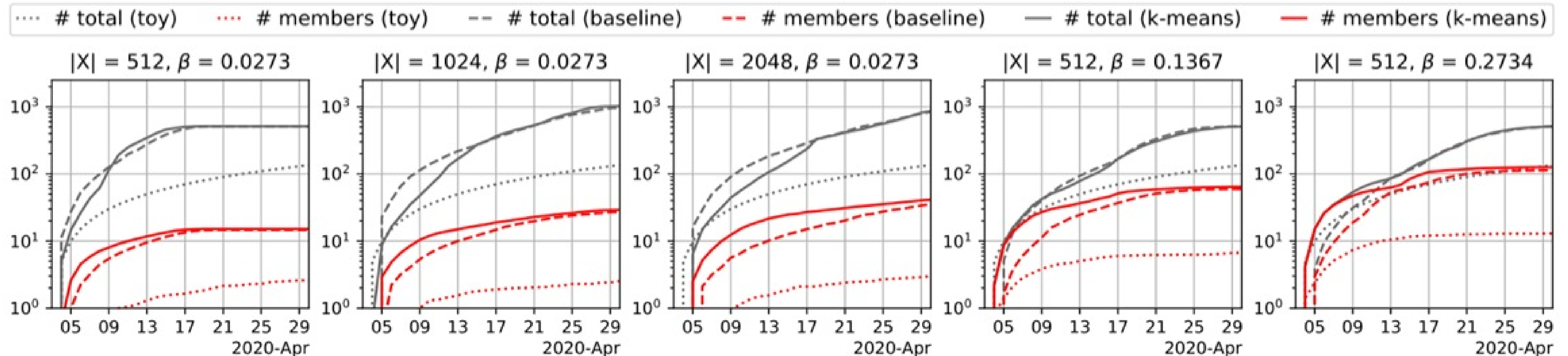
3. 实验评估

Scenario 2: Measurement of ad conversion revenue

Feature name(MI value –互信息)

age(0.0010), gender(0.0007), shopping_level(0.0002),
work(0.0002), consumption_ability(0.0001)

B 表示交集元素占比，假定一天实验可以执行5次协议。横轴：时间；纵轴：数值



- Set membership leakage in **the measurement of ad conversion revenue**, measured in # total and # members

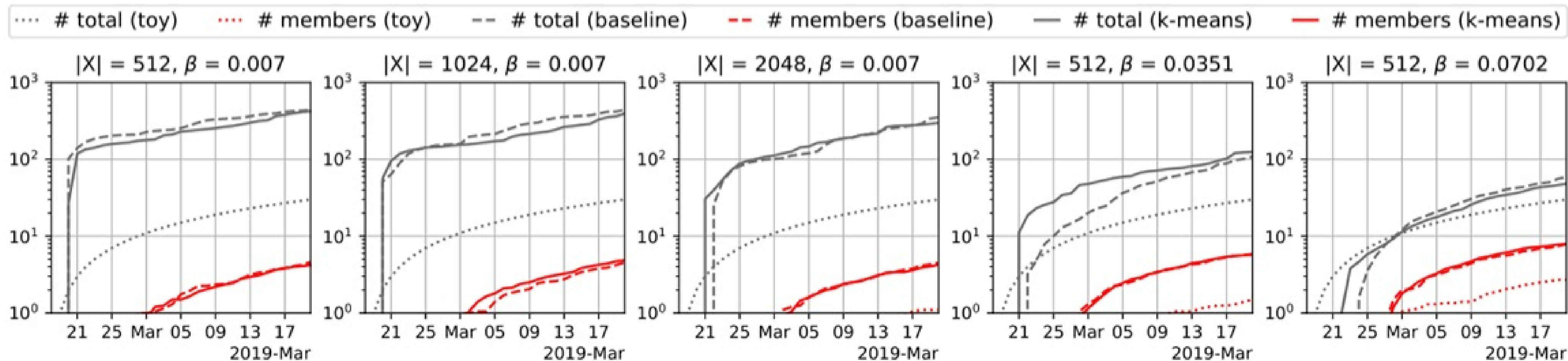
3. 实验评估

Scenario 3: Measurement of ad conversion lift

Feature name(MI value –互信息)

marriage_status(0.0013), education(0.0012),
consumption_ability(0.0012), age(0.0009)

B 表示交集元素占比，假定一天实验可以执行5次协议。横轴：时间；纵轴：数值



- Set membership leakage in **the measurement of ad conversion lift**, measured in # total and # members .
- 这个是交集极小的情况，只是延缓了命中交集元素的时间，但同样奏效

4. 现实影响

- COVID-19 contact tracing.
 - 攻击者可以猜测出谁得了新冠疾病。
- Measurement of ad conversion revenue.
 - 攻击者（即广告商）可以使用推断的成员来推断相关人员的兴趣。
- Measurement of ad conversion lift.
 - 广告商可以暗中利用发布者找到对广告感兴趣的人。

5. 防御措施

- 限制协议调用的次数
- 对交集大小进行审计
- 对攻击者输入集合大小进行审计
- 应用差分隐私

应用差分隐私足够吗？

Birds of a Feather Flock Together: How Set Bias Helps to Deanonimize You via Revealed Intersection Sizes.

USENIX Security, 2022

谢 谢

